BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC VĂN HIẾN

-----ꕥ*ꕥ-----

ThS. HỒ VĂN NGỌC

HỆ ĐIỀU HÀNH

(Tài liệu lưu hành nội bộ)

Năm 2022

MỤC LỤC

MỤC LỤC 1
DANH MỤC HÌNH 6
LỜI NÓI ĐẦU 8
DANH MỤC CÁC TỪ VIẾT TẮC 9
Chương 1: GIỚI THIỆU VÀ KIẾN TRÚC HỆ ĐIỀU HÀNH 10
1. **Một số khái niệm** 10
   a. **Hệ điều hành là gì?** 10
   b. **Chức năng chính của hệ điều hành** 11
   c. **Nhân (Kernel) của hệ điều hành** 11
   d. **Bootstrap program** 12
   e. **Interrupts**[1] 12
   f. **Cache** 13
2. **Kiến trúc của hệ điều hành** 13
   a. **Giới thiệu** 13
   b. **Các thành phần quan trọng của HĐH** 14
3. **Các địch vụ của HĐH** 15
4. **Giao diện người dùng** 16
5. **System calls (cuộc gọi hệ thống)** 17
6. **System Boot** 17

CÂU HỎI ÔN TẬP 19
ĐÁP ÁN 21
Chương 2: QUẢN LÝ TIẾN TRÌNH 22
1. **Khái niệm tiến trình** 22
2. **Các trạng thái của tiến trình** 24
3. **Quản lý tiến trình (Process Manager)** 25
4. **Luồng (tiểu trình)** 25
5. **Lập lịch tiến trình** 27
6. **Giao tiếp giữa các tiến trình** 28
7. **Các kiểu đa luồng** 30
   7.1. **Many-To-One** 31

7.2. One-To-One 31
7.3. Many-To-Many 32
CÂU HỎI ÔN TẬP 33
ĐÁP ÁN 34
Chương 3: LẬP LỊCH TIẾN TRÌNH 35
1. Giới thiệu về lập lịch 35
2. Một số thuật toán lập lịch 35
2.1. Thuật toán FCFS 36
2.2. Thuật toán SJF 39
2.3. Thuật toán PS 41
2.4. Thuật toán RR 44
CÂU HỎI ÔN TẬP 47
ĐÁP ÁN 49
BÀI TẬP 50
Chương 4: DEADLOCK 53
1. Giới thiệu về deadlock 53
2. Các phương thức quản lý deadlock 54
3. Ngăn chặn deadlock 54
3.1. Mutual exclusion 54
3.2. Hold and wait 54
3.3. No preemption 55
3.4. Circular wait 55
3.5. Tránh deadlock 57
3.6. Phát hiện deadlock 58
3.7. Phục hồi sau deadlock 58
CÂU HỎI ÔN TẬP 60
ĐÁP ÁN 62
Chương 5: QUẢN LÝ BỘ NHỚ 63
1. Một số khái niệm 63
2. Các kiểu địa chỉ 65
2.1. Địa chỉ vật lý (Physical Address) 65
2.2. Địa chỉ luận lý (Logical Address) 65

2.3. Chuyển đổi giữa địa chỉ vật lý và địa chỉ luận lý 66
3. Dynamic Loading 68
4. Overlay 68
5. Swapping 69
6. Cấp phát bộ nhớ 70
6.1. Static partition 70
6.2. Dynamic partition 73
CÂU HỎI ÔN TẬP 76
ĐÁP ÁN 78
Chương 6: QUẢN LÝ LƯU TRỮ 79
1. Một số khái niệm 79
2. Kiến trúc đĩa cứng (HDD) 82
3. Lập lịch truy xuất đĩa (HDD) 83
3.1. Điều kiện lập lịch đĩa: Dựa trên một số yếu tố sau. 83
3.2. Các thuật toán lập lịch đĩa: 84
CÂU HỎI ÔN TẬP 92
ĐÁP ÁN 94
TÀI LIỆU THAM KHẢO 95

*****

DANH MỤC HÌNH

Hình 1-1: Kiến trúc tổng quan của hệ điều hành 10
Hình 1-2: Mô hình 4 thành phần trong hệ thống máy tính 11
Hình 1-3: Minh họa kiến trúc nhân của hệ điều hành 12
Hình 1-4: Các thành phần quan trọng trong kiến trúc của hệ điều hành 14
Hình 1-5: Minh họa các dịch vụ của hệ điều hành 15
Hình 1-6: Hình ảnh giao diện dòng lệnh 16
Hình 1-7: Hình ảnh giao diện đồ họa 17
Hình 2-1: Các thành phần của tiến trình 22
Hình 2-2: Các thành phần của PCB 23
Hình 2-3: Mô hình chuyển đổi trạng thái của tiến trình 24
Hình 2-4: Thông tin của TCB 25
Hình 2-5: Các tiểu trình chia sẻ tài chuyên dùng chung 26
Hình 2-6: Mô hình Process Scheduling 28
Hình 2-7: Minh họa 2 mô hình IPC 29
Hình 2-8: Kernel level (space) and user level (space) in main memory 30
Hình 2-9: Đa luồng Many-To-One 31
Hình 2-10: Đa luồng One-To-One 31
Hình 2-11: Đa luồng Many-To-Many 32
Hình 4-1: Chu trình đợi tài nguyên 53
Hình 4-2: Biểu đồ cấp phát tài nguyên 54
Hình 4-3: Một ví dụ biểu đồ cấp phát tài nguyên 55
Hình 4-4: Một ví dụ biểu đồ RAG xuất hiện chu trình 55
Hình 4-5: Loại bỏ 2 đường cấp phát tài nguyên để tránh chu trình 56
Hình 5-1: Mô phỏng việc phân chia bộ nhớ 62
Hình 5-2: Mô phỏng không gian địa chỉ vùng nhớ cấp cho các tiến trình 63
Hình 5-3: Mô phỏng việc gán địa chỉ vật lý và luận lý. 64
Hình 5-4: Mô hình chuyển đổi địa chỉ ô nhớ. 66
Hình 5-5: Mô hình Swap-In và Swap-Out. 68

Hình 5-6: Bộ nhớ được chia thành các đoạn bằng nhau. 69
Hình 5-7: Bộ nhớ được cấp phát cho 3 xử lý. 70
Hình 5-8: Bộ nhớ được thành các phần cố định, kích thước khác nhau. 71
Hình 5-9: Hoạt động cấp phát vùng nhớ có kích thước cố định khác nhau. 72
Hình 5-10: Hoạt động cấp phát vùng nhớ động. 73
Hình 6-1: Kiến trúc thư mục 1 cấp. 79
Hình 6-2: Hình 6-2: Kiến trúc thư mục 2 cấp. 79
Hình 6-3: Kiến trúc thư mục đa cấp. 80
Hình 6-4: Kiến trúc thư mục trong Linux 80
Hình 6-5: Kiến trúc HDD. 81
Hình 6-6: Kiến trúc lá đĩa. 82
Hình 6-7: Tọa độ chuẩn bị cho sơ đồ di chuyển của đầu đọc. 84
Hình 6-8: Sơ đồ di chuyển của đầu đọc ghi. 84
Hình 6-9: Các quy ước trên lá đĩa. 87

*****

LỜI NÓI ĐẦU

Tập bài giảng này cung cấp cho sinh viên khoa công nghệ thông tin các kiến thức chung và nền tảng về hệ điều hành. giúp sinh viên biết được hệ điều hành ra đời như thế nào, nguyên lý và nhiệm vụ của chúng trong hệ thống máy tính hiện nay; Biết được những kiến thức này sẽ giúp sinh viên sử dụng và can thiệp vào máy tính dể dàng, có thể chuẩn đoán và phát hiện một số lỗi thường gặp để từ đó có hướng khắc phục và giải quyết.

DANH MỤC CÁC TỪ VIẾT TẮC

Từ viết tắt	Giải thích
CPU	Central Processing Unit
ISR	Interrupt Service Routine
HĐH	Hệ điều hành
CLI	Command-Line Interface
GUI	Graphical User Interface
BIOS	Basic Input/Output System
OS	Operating System
PCB	Process Control Block
TCB	Thread Control Block
CNTT	Công nghệ thông tin
API	Application Programming Interface
FIFO	First In First Out
HDD	Hard Disk Drive

Chương 1: GIỚI THIỆU VÀ KIẾN TRÚC HỆ ĐIỀU HÀNH

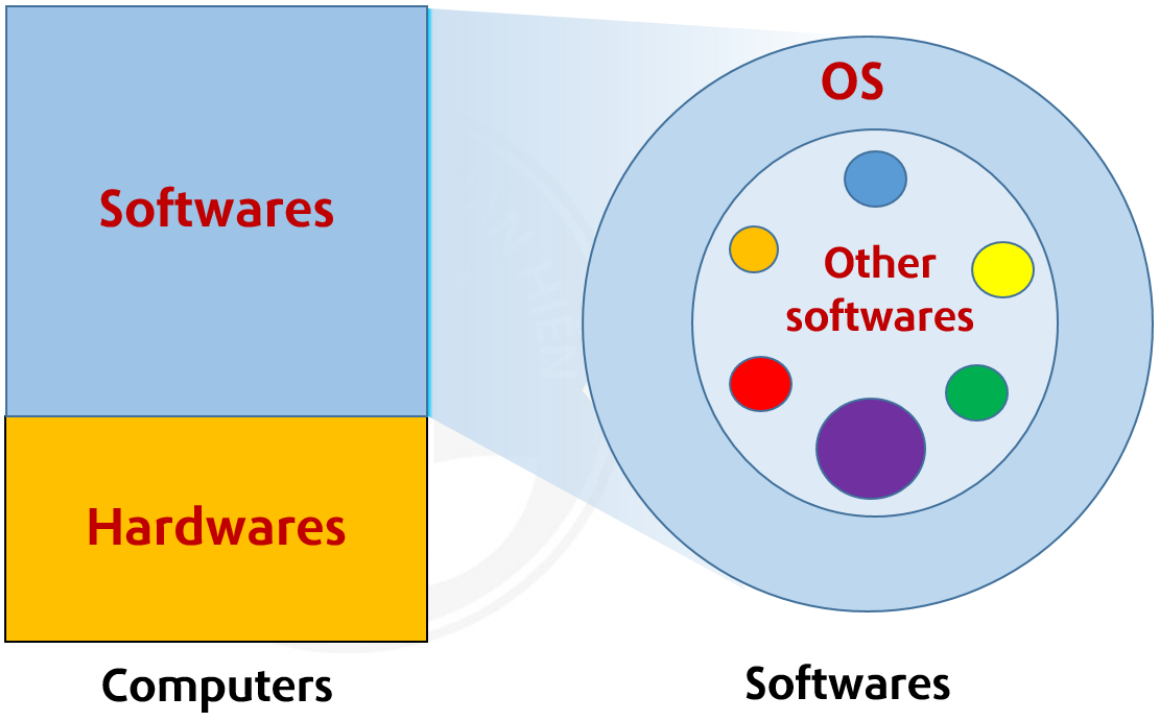
1. Một số khái niệm

a. Hệ điều hành là gì?

Hệ điều hành là phần mềm quan trọng nhất trong hệ thống máy tính. Nó không chỉ được cài đặt vào máy tính mà còn được cài đặt trên nhiều thiết bị khác.

Không có hệ điều hành, máy tính không thể hoạt động được. Có thể nói: Nếu xem phần cứng là thể xác thì hệ điều hành chính là linh hồn của máy.

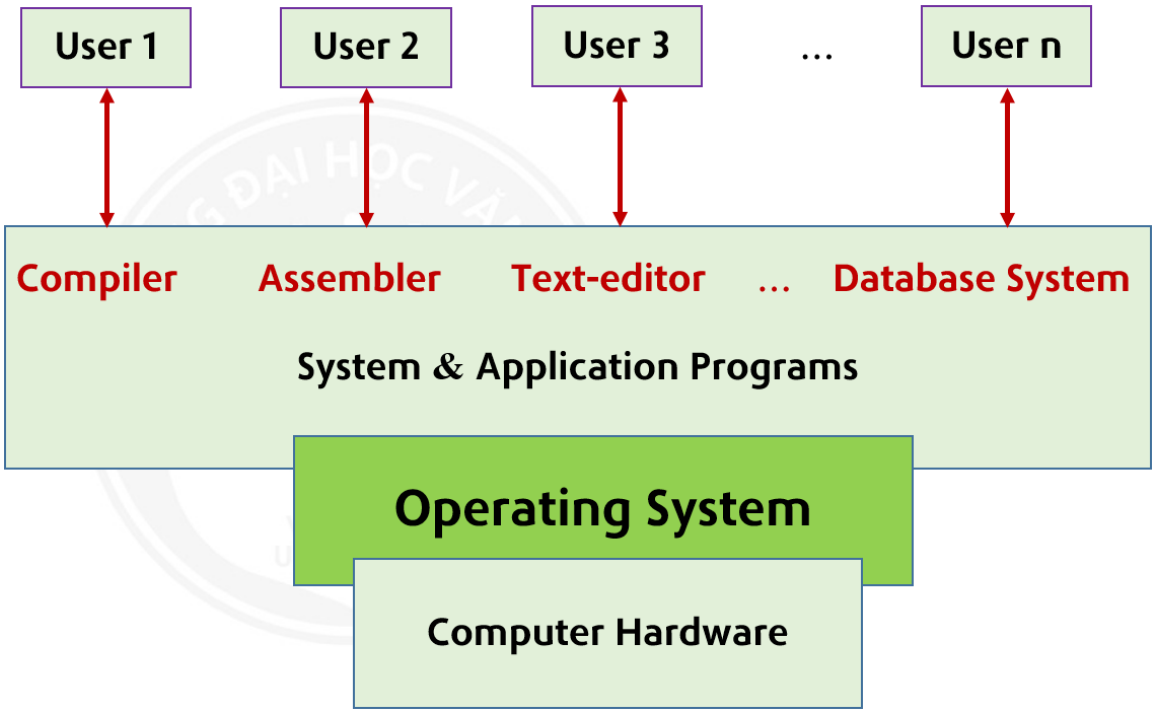
Trong phạm vi học phần này, chúng ta quan tâm chủ yếu đến các hệ điều hành hoạt động trên hệ thống máy tính (desktop, laptop) và các chức năng có trong hầu hết các hệ điều hành. Sau đây là hình ảnh minh học kiến trúc tổng quan của hệ điều hành.



Hình 1-1: Kiến trúc tổng quan của hệ điều hành

Hệ điều hành là một phần mềm được tạo ra với rất nhiều chức năng, nhằm mục đích quản lý mọi hoạt động của hệ thống máy tính và người sử dụng; Cụ thể, nó quản lý tất cả phần cứng và các phần mềm khác trong hệ thống máy tính.

Có 4 thành phần quan trọng trong một hệ thống máy tính; Trong đó hệ điều hành đóng vai trò trung gian giữa phần cứng máy tính với các phần mềm khác và với người sử dụng; Đồng thời nó cung cấp môi trường cho phép người sử dụng phát triển và thực thi các ứng dụng.



Hình 1-2: Mô hình 4 thành phần trong hệ thống máy tính

b. Chức năng chính của hệ điều hành

Hệ điều hành có hai chức năng chính:

+ Cấp phát và theo dõi tài nguyên

+ Thực thi các chính sách (policies)

Ví dụ: chức năng quản lý bộ nhớ (Memory Management)

+ Theo dõi tình trạng không gian bộ nhớ của máy tính

+ Cấp phát đúng số lượng ô nhớ cho các xử lý mới phát sinh

+ Cấp lại vùng nhớ khi thích hợp

+ Thực thi các chính sách liên quan đến việc sử dụng bộ nhớ

c. Nhân (Kernel) của hệ điều hành

Là thành phần trung tâm của hầu hết các hệ điều hành máy tính. Mỗi HĐH có cấu trúc nhân khác nhau. Các nhân khác nhau thực hiện các tác vụ của hệ điều hành theo các cách khác nhau.

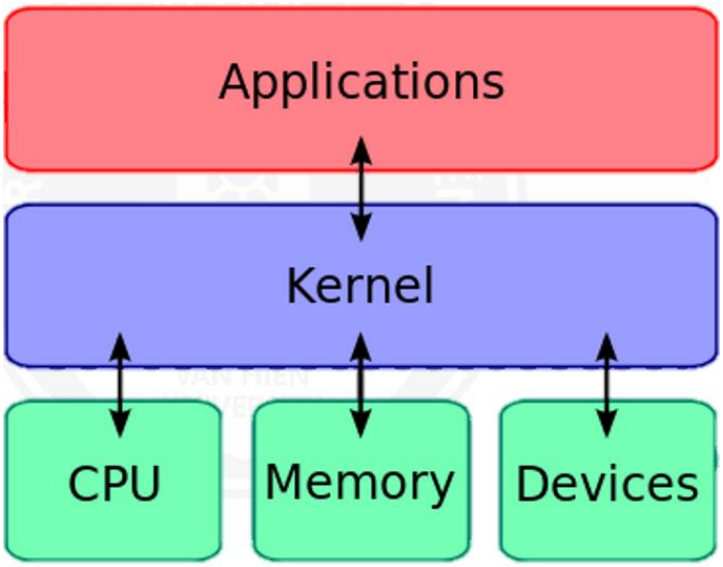
Ví dụ: Nhân của hệ điều hành Linux khác nhân của hệ điều hành Windows.

Nhiệm vụ của nhân hệ điều hành:

+ Quản lý các tài nguyên hệ thống

+ Liên kết giữa các thành phần phần cứng và phần mềm

Sau đây là hình ảnh minh họa kiến trúc nhân của hệ điều hành:



Hình 1-3: Minh họa kiến trúc nhân của hệ điều hành

d. Bootstrap program

Đây là một chương trình nhỏ, được lưu trữ trong ROM (hay EPROM). Nó được kích hoạt khi boot hay reboot máy.

Mục đích chính của chương trình này là:

+ Khởi tạo tất cả các thành phần của hệ thống

+ Load nhân của hệ điều hành vào bộ nhớ và thực thi

Khi mở máy, chúng ta thường thấy trên màn hình máy tính xuất hiện nền đen, chữ trắng trước khi biểu tượng của hệ điều hành xuất hiện. Đó là thông tin của chương trình bootstrap đang chạy.

e. Interrupts

Còn gọi là ***ngắt***; là sự kiện dừng công việc hiện tại của CPU, buộc CPU thực hiện một việc nào đó rồi mới quay trở lại thực hiện tiếp công việc cũ. Khi công việc hiện tại bị buộc dừng, trạng thái của nó được lưu giữ lại để chờ CPU quay lại thực hiện tiếp. Một chương trình chuyên phục vụ các ngắt nằm trong hệ điều hành gọi là *trình xử lý ngắt* (ISR – Interrupt Service Routine).

Trong quá trình hoạt động của máy tính, rất nhiều sự kiện (events) phát sinh. Nguồn gốc phát sinh có thể là:

+ Bên trong hệ thống máy tính

+ Từ các thiết bị ngoại vi

+ Từ users

Sự xuất hiện của một sự kiện thường được báo hiệu bằng ngắt. Ngắt được chia làm 2 loại:

+ **Ngắt đồng bộ**: Ngắt được tạo ra bởi bộ điều khiển trung tâm (CPU). Ngắt được gọi là đồng bộ vì trong quá trình CPU thực thi các câu lệnh, nó gọi ngắt sau khi kết thúc việc thực thi của một lệnh. CPU là thiết bị chủ động tạo ra ngắt.

+ **Ngắt không đồng bộ**: Ngắt được tạo ra bởi các thiết bị phần cứng khác, và được tạo ra ở bất kỳ thời điểm nào mà không liên quan đến các câu lệnh hiện tại đang được CPU thực thi. CPU không là thiết bị chủ động tạo ra ngắt.

Phần cứng phát ra các ngắt dưới dạng tín hiệu (*signal*), gửi đến CPU, thông qua system bus.

Phần mềm phát ra các ngắt bằng cách thực thi một cơ chế đặc biệt gọi là *system call* (hay còn gọn là monitor call).

Khi CPU nhận được ngắt, nó chuyển các ngắt này đến các dịch vụ tương ứng (để xử lý).

f. Cache

Là một vùng nhớ đệm có kích thước nhỏ nhưng tốc độ truy xuất rất nhanh. Mục đích của cache là lưu trữ thông tin tạm thời nhằm tăng tốc xử lý.

Cache có thể là phần cứng hay phần mềm; Nó thường được sử dụng để cải thiện hiệu suất cho các ứng dụng và thiết bị. Một ví dụ sử dụng cache là các website có lượt truy cập cao.

2. Kiến trúc của hệ điều hành

a. Giới thiệu

Hệ điều hành là phần mềm lớn nhất được cài đặt vào trong máy tính. Nó được trang bị rất nhiều chức năng để có thể quản lý được mọi hoạt động của hệ thống, bao gồm: phần cứng, phần mềm và người sử dụng. Do đó, hệ điều hành có kiến trúc phức tạp.

Hiện nay có rất nhiều loại HĐH đang được sử dụng (Windows, Linux, Mac OS, …). Mỗi loại hệ điều hành có kiến trúc khác nhau (do được tạo ra từ các teams khác nhau) và cách thức hoạt động của chúng cũng khác nhau. Ví dụ: kiến trúc HĐH Windows khác kiến trúc HĐH Linux.

b. Các thành phần quan trọng của HĐH

HĐH có rất nhiều thành phần. Sau đây là các thành phần rất quan trọng mà hầu hết các HĐH đều có:

+ Memory Manager: Quản lý bộ nhớ

+ Process Manager: Quản lý tiến trình

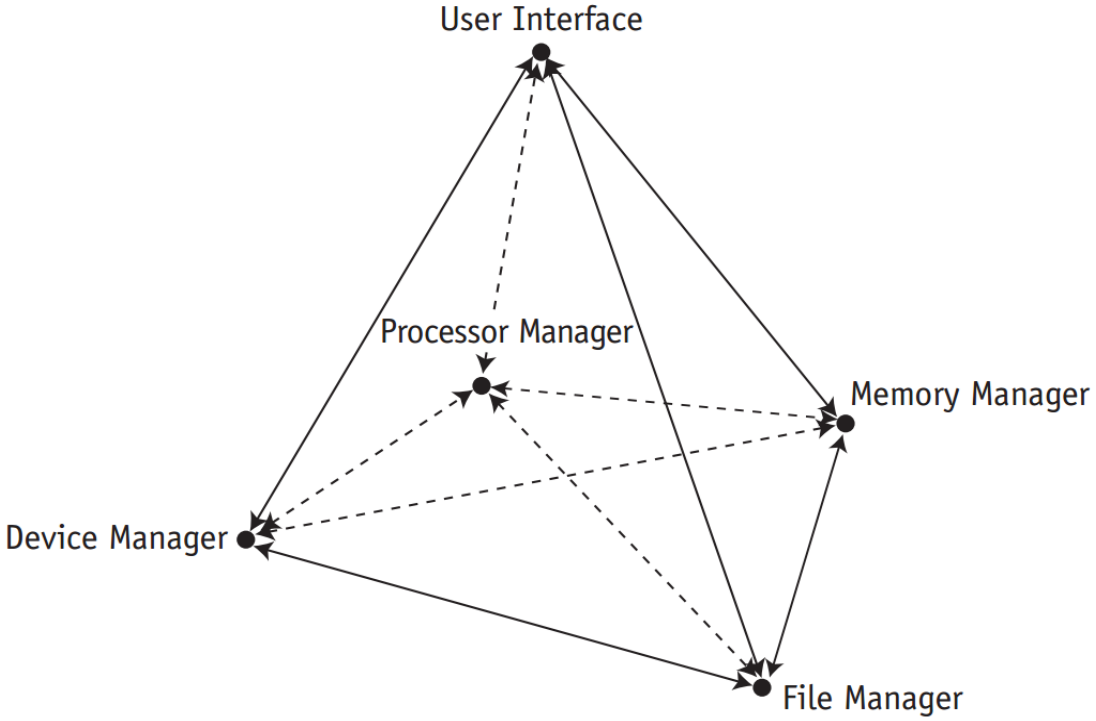
+ Device Manager: Quản lý thiết bị

+ File Manager: Quản lý tập tin

+ User Interface: Quản lý giao diện người dùng

Chúng ta sẽ đề cập các thành phần trên trong các chương sau của học phần này.

Các thành phần trong kiến trúc của HĐH có thể được biểu diễn qua hình ảnh sau:



Hình 1-4: Các thành phần quan trọng trong kiến trúc của hệ điều hành

Mỗi thành phần nêu trên còn gọi là sub-system. Mỗi sub-system phụ trách một số chức năng riêng; Tuy nhiên mỗi sub-system có giao tiếp với các sub-system khác.

3. Các địch vụ của HĐH

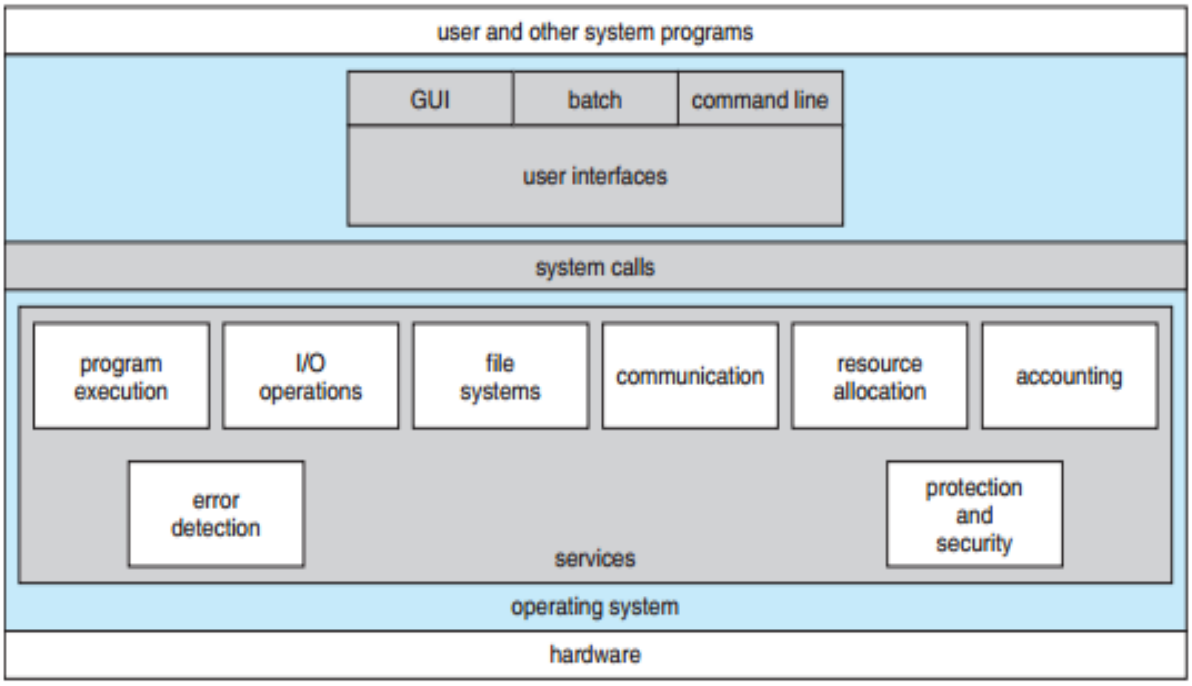
HĐH cung cấp rất nhiều dịch vụ phục vụ cho:

+ Người dùng (users)
+ Xử lý (processes)
+ Các hệ thống khác (other systems)

Sau đây liệt kê một số dịch vụ:

+ Program Execution: Thực thi chương trình
+ IO Operations: Các thao tác nhập xuất
+ File Systems: Hệ thống tập tin
+ Communication: Truyền thông, giao tiếp
+ Resource Allocation: Cấp phát tài nguyên
+ Accounting: Thống kê, kiểm toán
+ Error Detection: Phát hiện lỗi
+ Protection and Security: An toàn và bảo mật
+ …

Sau đây là hình ảnh minh họa các dịch vụ của HĐH:



Hình 1-5: Minh họa các dịch vụ của hệ điều hành

4. Giao diện người dùng

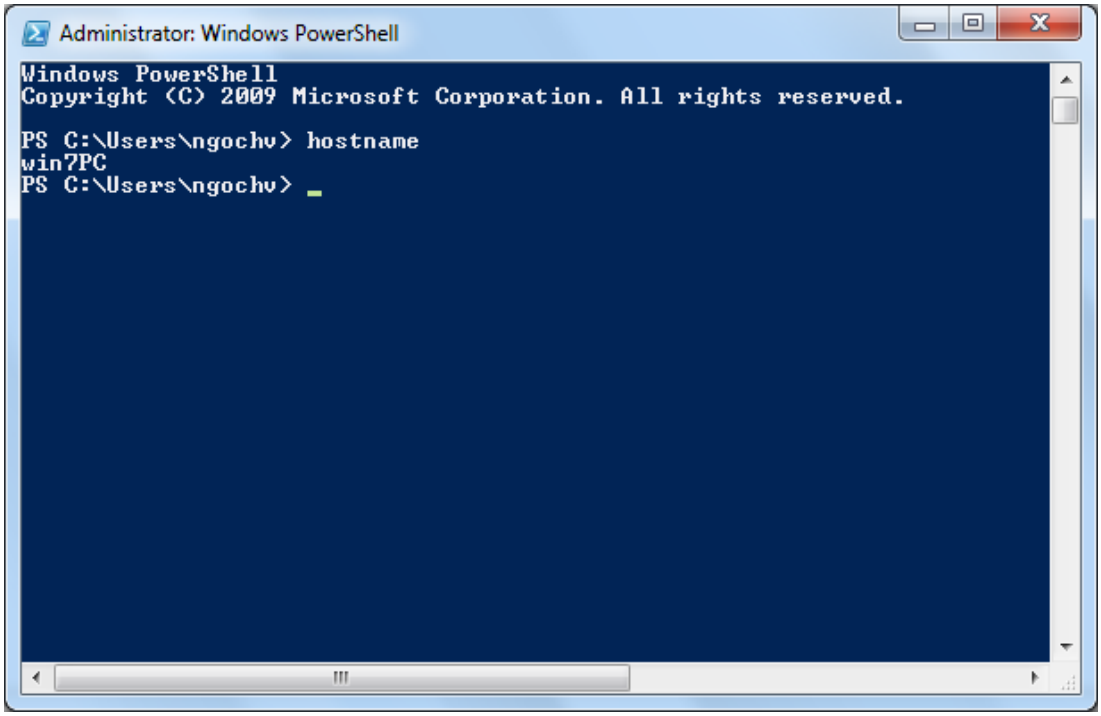
Có hai loại giao diện của HĐH:

+ Giao diện dòng lệnh (CLI - Command-Line Interface)

+ Giao diện đồ họa (GUI - Graphical User Interface)

Tùy theo nhu cầu và khá năng của người sử dụng mà ta chọn lựa loại giao diện thích hợp. Ví dụ với người dùng cuối (end users), giao diện đồ họa thích hợp nhất vì dễ sử dụng; Với các cơ quan có nhu cầu cung cấp các dịch vụ hoạt động thường xuyên trong thời gian dài thì giao diện dòng lệnh thích hợp hơn.

a. Giao diện dòng lệnh



Hình 1-6: Hình ảnh giao diện dòng lệnh

Một số HĐH sử dụng giao diện dòng lệnh làm giao diện chính, ví dụ: Core Linux.

Một số HĐH cung cấp giao diện dòng lệnh như là một ứng dụng, ví dụ:

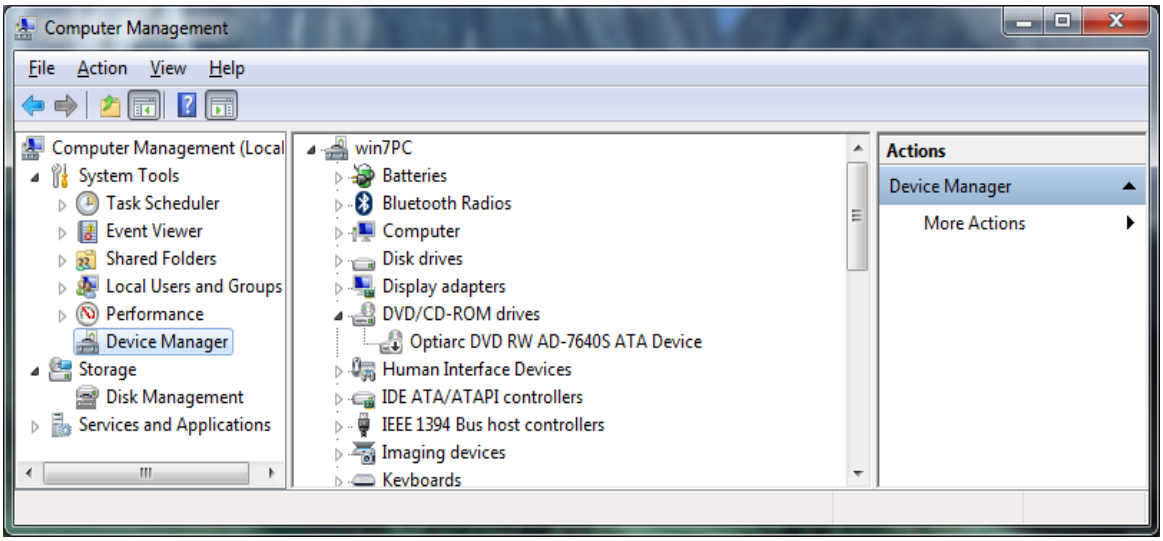
+ HĐH Windows có ứng dụng dòng lệnh: Command Prompt, Power Shell

+ HĐH Linux (GUI) có ứng dụng dòng lệnh: Terminal, Bash Shell

Các máy chủ Linux thường sử dụng giao diện dòng lệnh để tăng khả năng ổn định, tăng tốc xử lý và giảm tải do hệ thống giao diện.

b. Giao diện đồ họa

Giúp người dùng dể sử dụng các chức năng của máy tính thông qua con chuộc (mouse) và bàn phím (keyboard). Tuy nhiên, nhiều chức năng chuyên sâu có thể không có trong hệ thống giao diện. Sau đây là một hình ảnh giao diện đồ họa trong Windows:



Hình 1-7: Hình ảnh giao diện đồ họa

5. System calls (cuộc gọi hệ thống)

HĐH cung cấp cách thức giao tiếp giữa một xử lý (hay chương trình) với các dịch vụ của nó. Mỗi cách thức giao tiếp được gọi là một system call và có thể bao gồm nhiều chức năng; Có thể phân các system calls thành 6 loại như sau:

+ Process Control: Điều khiển xử lý

+ File management: Quản lý tập tin

+ Device management: Quản lý thiết bị

+ Information Maintenance: bảo trì thông tin

+ Communication: Truyền thông, giao tiếp

+ Protection: Bảo vệ

6. System Boot

Là xử lý khởi động máy tính bằng cách load nhân (kernel) của HĐH vào bộ nhớ và thực thi. Chức năng này nhờ một chương trình nhỏ gọi là bootstrap program (hay bootstrap loader). Chương trình nhỏ này có nhiệm vụ:

+ Định vị Kernel (được lưu trên đĩa cứng)

+ Load kernel vào bộ nhớ (chuẩn bị thực thi)

+ Khởi động và chạy HĐH

Việc xử lý boot trải qua các giai đoạn sau:

+ Chương trình trong ROM được khởi động

+ Kiểm tra BIOS

+ Kiểm tra RAM

+ Khởi động CPU

+ Kiểm tra Hardware

+ Tìm và Load HĐH

Để đảm bảo hệ thống boot thành công, 3 thành phần sau phải hoạt động thích hợp:

+ BIOS

+ OS

+ Hardwares

*****

CÂU HỎI ÔN TẬP

Câu 1:

Bốn thành phần quan trọng trong hệ thống máy tính là:

a. Phần cứng, HĐH, các chương trình ứng dụng, các phần mềm khác
b. Phần cứng, người sử dụng, HĐH, các phần mềm khác
c. Phần mềm, phần cứng, HĐH, các thiết bị ngoại vi
d. HĐH, phần cứng, nhân (Kernel), HĐH

Câu 2:

Trong kiến trúc 4 thành phần của hệ thống máy tính,

a. HĐH nằm giữa phần cứng và các phần mềm khác
b. HĐH nằm chung với các phần mềm khác
c. HĐH nằm giữa người sử dụng và các phần mềm khác
d. HĐH nằm ở cuối cùng trong kiến trúc

Câu 3:

Khái niệm "Kernel" chính là:

a. HĐH
b. Các chương trình ứng dụng quan trọng
c. Lõi của HĐH
d. Phần cứng máy tính

Câu 4:

Bootstrap program là chương trình dùng để khởi động máy tính. Nó nằm trong:

a. RAM
b. ROM
c. CPU
d. HDD/SSD

Câu 5:

HĐH cung cấp cơ ché giao tiếp giữa một xử lý (hay chương trình) với các phần mềm khác, với phần cứng máy tính và các dịch vụ; Cơ chế đó gọi là:

a. Communication (giao tiếp)
b. Connection (kết nối)
c. Interface (giao diện)
d. System call (lời gọi hệ thống)

*****

ĐÁP ÁN

Câu 1: b

Câu 2: a

Câu 3: c

Câu 4: b

Câu 5: d

*****

Chương 2: QUẢN LÝ TIẾN TRÌNH

1. Khái niệm tiến trình

Một tiến trình (process) là một thể hiện (instance) của một chương trình đang thực thi. Do đó, một chương trình chưa thực thi thì không phải là một tiến trình.

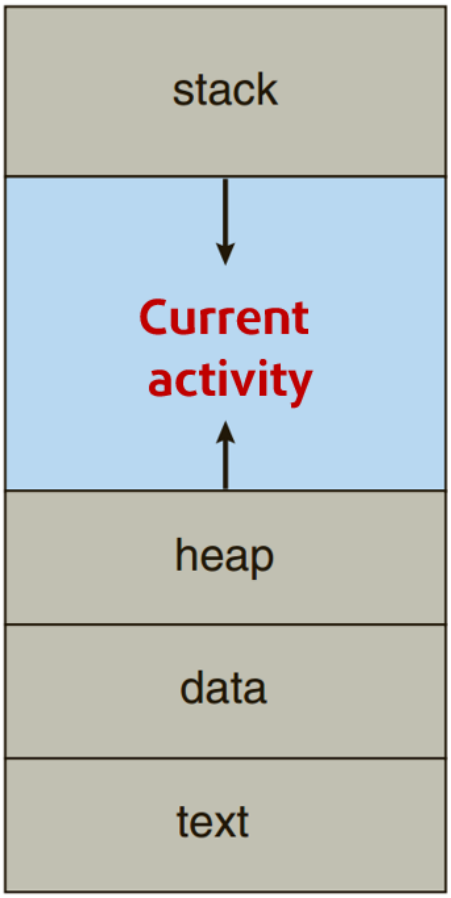
Một chương trình trở thành một tiến trình khi nó được load vào bộ nhớ và thực thi.

Trong hệ thống máy tính có rất nhiều tiến trình. Có thể phân loại các tiến trình làm hai loại:

+ Các tiến trình của HĐH

+ Các tiến trình của người sử dụng

Mỗi tiến trình bao gồm nhiều thành phần:



Hình 2-1: Các thành phần của tiến trình

Chúng ta lần lượt xem xét từng thành phần của tiến trình:

+ **Current activity**: Thành phần này có 2 chức năng quan trọng là:

- Program counter: Còn có tên gọi khác là *con trỏ lệnh* (instruction pointer), chỉ đến lệnh cần thực thi kế tiếp (trong bộ nhớ).
- Processor registers: Là các thanh ghi (bộ nhớ) của CPU, dùng để lưu trữ, phục vụ việc tính toán và thực thi lệnh.

+ **Stack**: Dùng để chứa dữ liệu tạm thời, bao gồm:

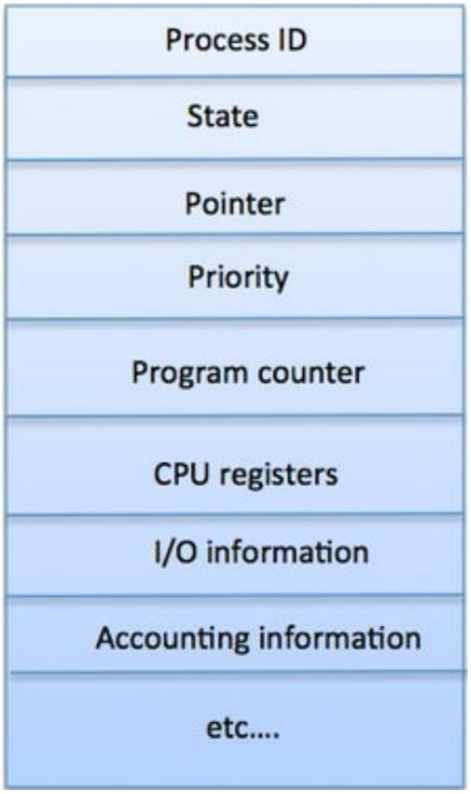
- Function parameters: Tham số của hàm
- Return addresses: Địa chỉ trả về từ hàm
- Local variables: Các biến cục bộ

+ **Text section**: Chứa code của chương trình

+ **Data section**: Chứa các biến toàn cục

+ **Heap**: Chứa vùng nhớ cấp phát động trong suốt thời gian hoạt động của chương trình

Mỗi tiến trình hoạt động có một khối điều khiển tiến trình (PCB - Process Control Block). PCB còn có tên gọi khác là *task control block*. Đây là một cấu trúc dữ liệu nằm bên trong kernel của HĐH, Chứa nhiều thông tin dùng để quản lý hoạt động của tiến trình. Mô tả các thành phần của PCB thông qua hình sau:



Hình 2-2: Các thành phần của PCB

Như vậy, mỗi tiến trình có một không gian địa chỉ, một con trỏ lệnh, một tập các thanh ghi và stack (ngăn xếp) riêng. Tiến trình có thể cần đến một số tài nguyên như CPU, bộ nhớ chính (RAM), các tập tin và thiết bị nhập/xuất. Hệ điều hành sử dụng bộ điều phối (scheduler) để quyết định thời điểm cần dừng hoạt động của tiến trình đang xử lý và lựa chọn tiến trình tiếp theo cần thực hiện.

Lưu ý: Thông tin trong các PCBs có thể khác nhau tùy thuộc vào từng loại HĐH.

2. Các trạng thái của tiến trình

Mỗi tiến trình, từ lúc phát sinh cho đến khi kết thúc có thể trải qua nhiều trạng thái khác nhau. Sau đây liệt kê các trạng thái của tiến trình.

+ **new** : tiến trình được tạo

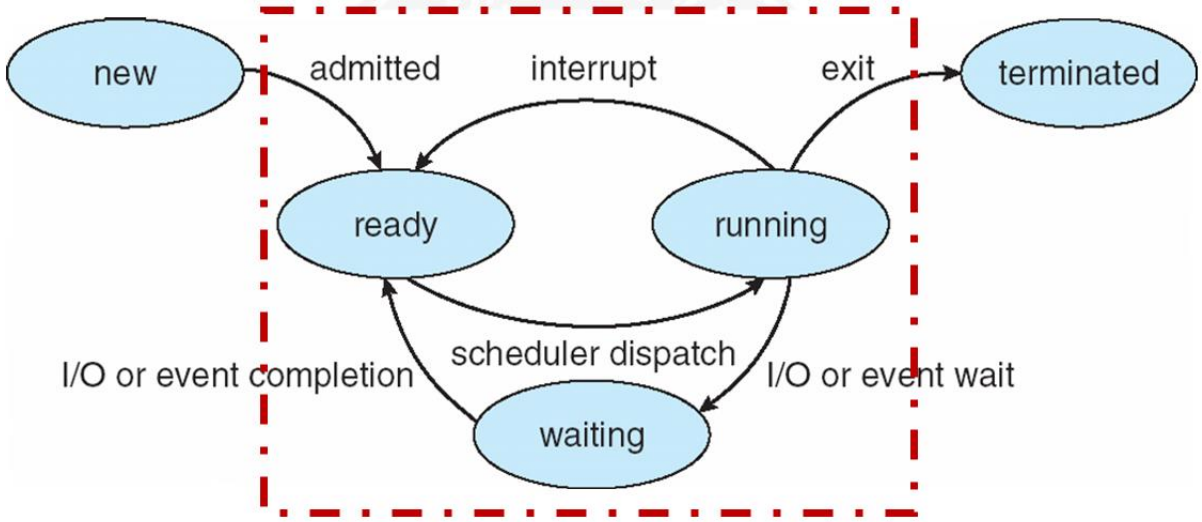
+ **running** : tiến trình được thực thi

+ **waiting** : tiến trình đang đợi sự kiện nào đó

+ **ready** : tiến trình sẵn sàng hoạt động

+ **terminate** : tiến trình kết thúc

Mô hình chuyển đổi trạng thái của tiến trình:



Hình 2-3: Mô hình chuyển đổi trạng thái của tiến trình

Trong hình 2-3:

+ Hình elip: Trạng thái của tiến trình

+ Mũi tên: Hướng chuyển đổi trang thái

3. Quản lý tiến trình (Process Manager)

Đây là một trong các chức năng quan trọng của HĐH. Chức năng này bao gồm hai chức năng quản lý (sub-manager):

+ Job Scheduler

+ Process Scheduler

Một ứng dụng đang chạy có thể xem như là một Job; Ví dụ chúng ta mở ứng dụng Microsoft Word thì Word xem như là một Job.

Mỗi Job có thể tạo ra một hay nhiều tiến trình; Ví dụ trong ứng dụng Word ta sử dụng các chức năng như: Kiểm lỗi chính tả, tìm kiếm một từ, định dạng văn bản, … Mỗi chức năng này xem như là một tiến trình.

Một chức năng quan trọng của quản lý tiến trình là lập lịch tiến trình, sẽ được đề cập chi tiết trong chương: Lập lịch tiến trình.

4. Luồng (tiểu trình)

Một tiến trình có thể tạo ra một hay nhiều tiểu trình (tiến trình con) hay gọi là luồng (threads), mỗi tiểu trình thực hiện một chức năng nào đó. Nhiều tiểu trình có thể thực thi đồng thời bằng cách chia sẻ CPU và một số tài nguyên khác. Các tiểu trình trong cùng một tiến trình dùng chung không gian địa chỉ tiến trình nhưng có con trỏ lệnh, tập các thanh ghi và stack riêng. Một tiểu trình cũng có thể tạo lập các tiểu trình con.

Các tiểu trình cũng nhận các trạng thái khác nhau tương tự như một tiến trình. Mô hình chuyển đổi trạng thái của tiến trình (hình 2-3) cũng áp dụng được cho tiểu trình.

Mỗi tiểu trình có một cấu trúc dữ liệu đi kèm gọi là: *Thread Control Block* (TCB). Thông tin và các thành phần của TCB được mô tả trong hình sau:

```
Thread identification
Thread state
CPU information:
        Program counter
        Register contents
Thread priority
Pointer to process that created this thread
Pointers to all other threads created by this thread
```

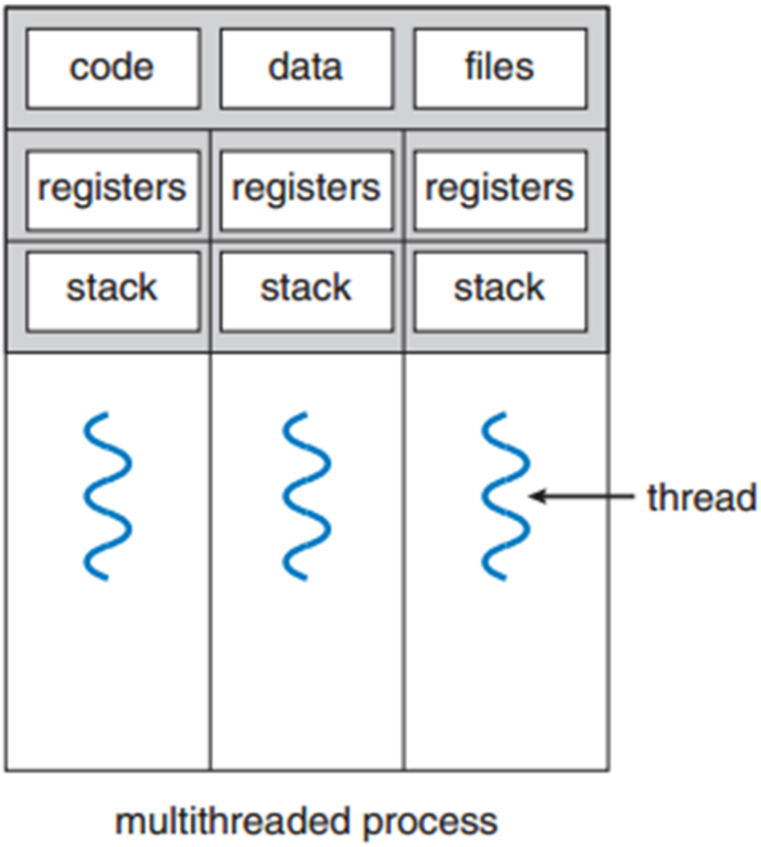
Hình 2-4: Thông tin của TCB

Sau đây là hình ảnh cho thấy các luồng của một tiến trình có thể chia sẻ thông tin cho nhau. Thông tin chia sẻ bao gồm:

+ code section

+ data section

+ files secsion



Hình 2-5: Các tiểu trình chia sẻ tài chuyên dùng chung

Ví dụ:

+ Hoạt động của Web browser là 1 process, có các threads:

- Hiển thị **pictures**
- Hiển thị **text**
- Nhận **data** từ network

5. Lập lịch tiến trình

Trong hệ thống máy tính, nếu tài nguyên được nhiều tiến trình yêu cầu thì sẽ xuất hiện hàng đợi tiến trình; Có nhiều hàng đợi (queues) trong hệ thống. Để phân phối tài nguyên hợp lý, tránh tình trạng deadlock, cần phải có chức năng lập lịch tiến trình.

Sau đây là một số hàng đợi cần được lập lịch:

- **job queue**: chứa tất cả các processes được đưa vào hệ thống (được kích hoạt để chạy)
- **ready queue**: chứa các processes đã được đưa vào bộ nhớ và sẵn sàng đợi tới lượt để thực thi (tương tự như một **linked list**)
- **device queue**: chứa các processes đang cùng đợi một thiết bị IO nào đó. **Ví dụ**: Print queue; mỗi thiết bị đều có queue của riêng nó.
- Ngoài ra, hệ thống có thể có **nhiều hàng đợi khác**.

Khi có nhiều tiến trình đang hoạt động → chúng được đưa vào Processes Queue. Processes Scheduler sẽ điều phối các tiến trình này, quyết định tiến trình nào thực thi; cụ thể:

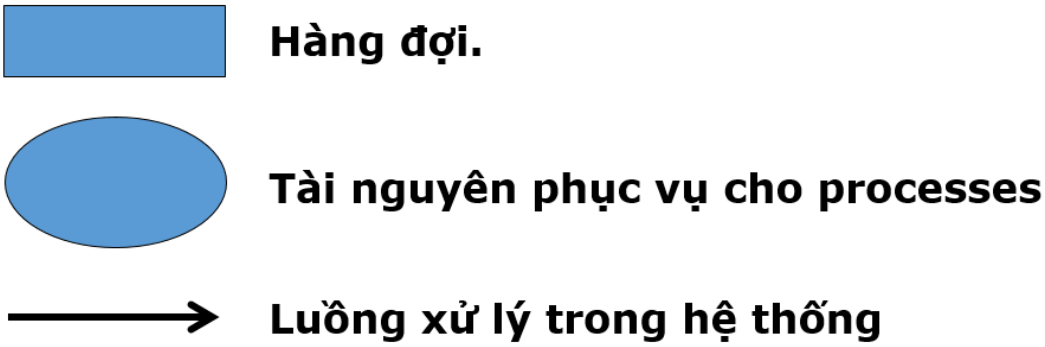
- Process nào sẽ nhận CPU
- Khi nào nhận CPU
- Sử dụng CPU trong bao lâu
- …

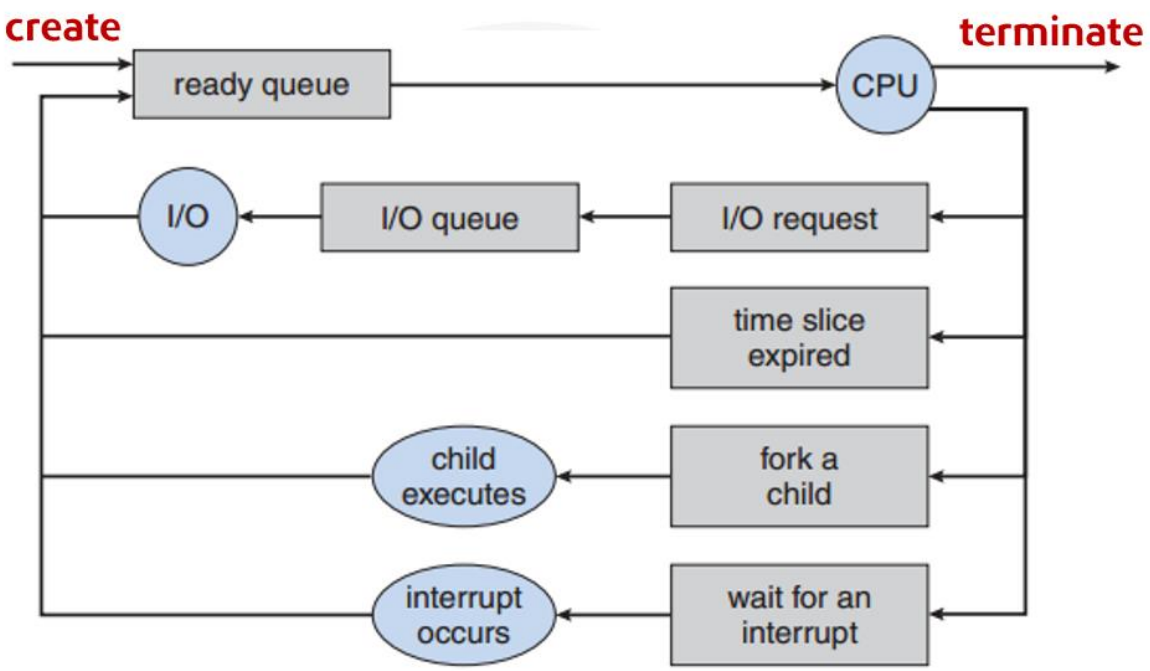
Khi một process được cấp phát CPU → nó thực thi trong một thời gian nhất định và cuối cùng rơi vào một trong các trạng thái sau:

- **Kết thúc**
- **Bị ngắt** (interrupted)
- **Đợi** một sự kiện nào đó xuất hiện như: I/O Request, tài nguyên, …

Mô hình sau đây mô tả chung cho process scheduling

Ký hiệu dùng trong mô hình:





Hình 2-6: Mô hình Process Scheduling

Giải thích mô hình:

+ Một process **mới được tạo** ra sẽ được đặt vào **ready queue**.

+ Một khi process **được cấp phát CPU, nó sẽ thực thi**, sau đó một trong các sự kiện sau có thể phát sinh:

- Process yêu cầu **I/O request → I/O queue**
- **Hết thời gian** thực thi **→ Ready queue**
- Process tạo ra **new child process** và đợi cho các child processes này kết thúc.
- Process bị buộc ra khỏi CPU (**interrupted**) và sẽ được đặt lại vào **ready queue.**

6. Giao tiếp giữa các tiến trình

Các processes thực thi đồng thời trong HĐH có thể là:

- Các process **độc lập**
- Các process **cộng tác**

Mục đích các processes cộng tác:

- Chia sẻ thông tin
- Tăng tốc tính toán
- Chia nhỏ xử lý (Modularity)

→ Cần có cơ chế quản lý việc cộng tác.

Cơ chế IPC (interprocess communication) cho phép các processes cộng tác trao đổi dữ liệu cho nhau. Có 2 mô hình IPC cơ bản:

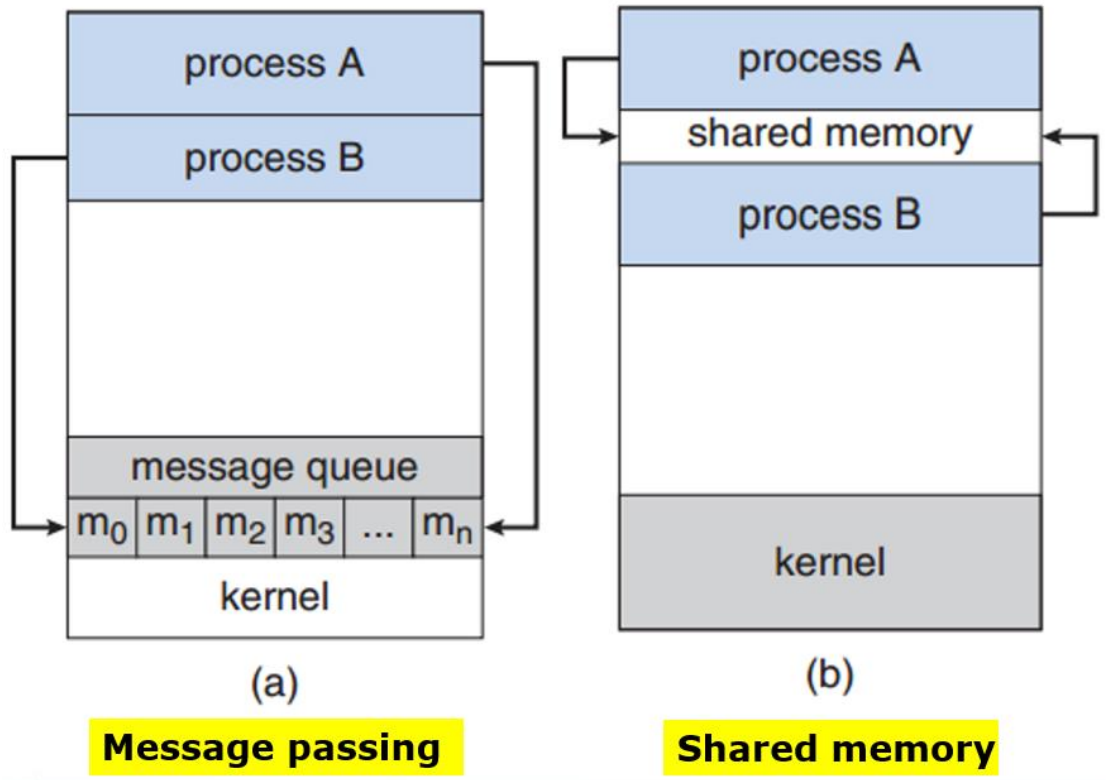
Mô hình 1: Shared memory (Hình 2-7 b).

- Vùng bộ nhớ được share bởi các processes cộng tác được thiết lập.
- Các processes có thể trao đổi thông tin bằng cách đọc và ghi dữ liệu vào vùng nhớ shared này.
- Có thể nhanh hơn Message Passing
- Cần nhiều thời gian của kernel hơn

Mô hình 2: Message passing (Hình 2-7 a).

- Các processes cộng tác trao đổi cho nhau các messages (shared).
- Hữu dụng khi trao đổi dữ liệu số lượng nhỏ
- Dể triển khai trên các hệ thống phân tán khi so sánh với Shared Memory.

Nhiều hệ thống có thể sử dụng cả 2 mô hình. Hình vẽ sau đây cho thấy 2 mô hình IPC:

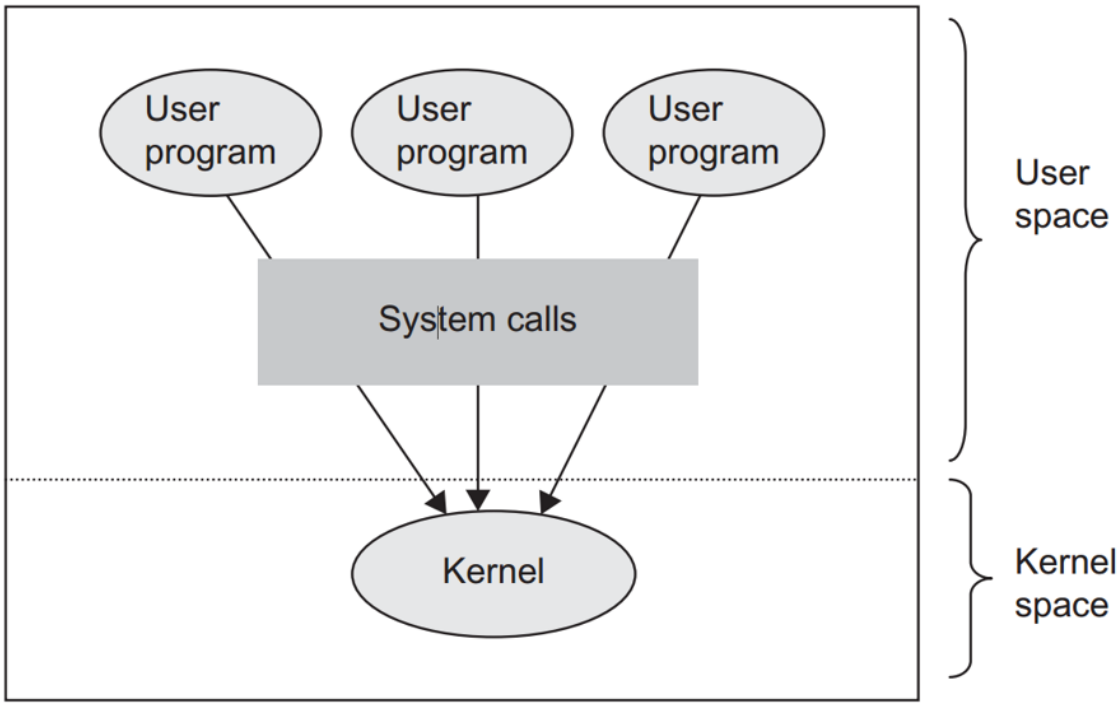


Hình 2-7: Minh họa 2 mô hình IPC

7. Các kiểu đa luồng

Trong HĐH có 2 kiểu đa luồng sau:

- **Kernel level** (kernel threads): được hỗ trợ và **quản lý trực tiếp** bởi HĐH.
- **User level** (user threads): được hỗ trợ ở **phía trên** của kernel threads (Hình 2-8).



Hình 2-8: Kernel level (space) and user level (space) in main memory

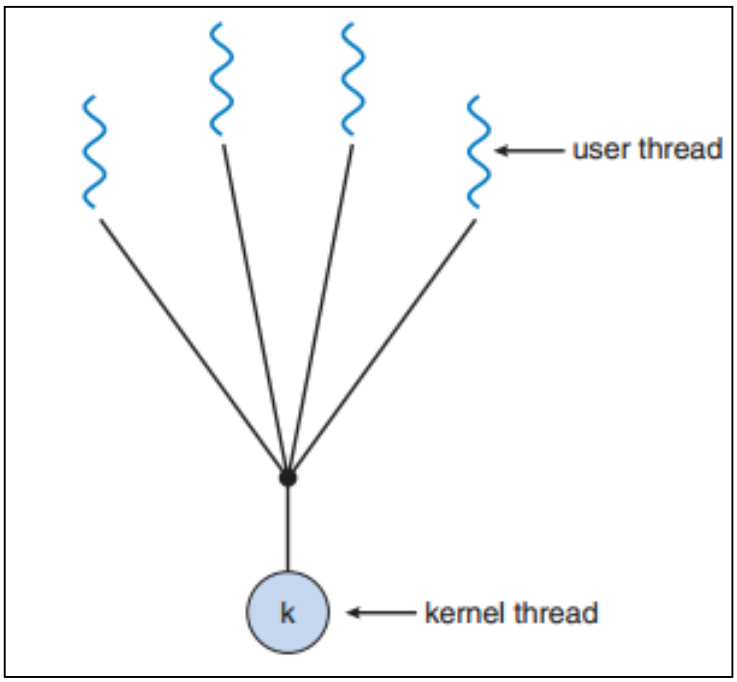
HĐH có thư viện hỗ trợ luồng là **Threads library** chuyên cung cấp **API** cho các lập trình viên để tạo và quản lý threads. Các công cụ của **API** bao gồm:

- Thread cancellation
- Signal handling (synchronous/asynchronous)
- Handling thread-specific data
- Scheduler activations
- …

Trong HĐH, có 3 kiểu đa luồng sau:

- **Many-to-One**
- **One-to-One**
- **Many-to-Many**

7.1. Many-To-One

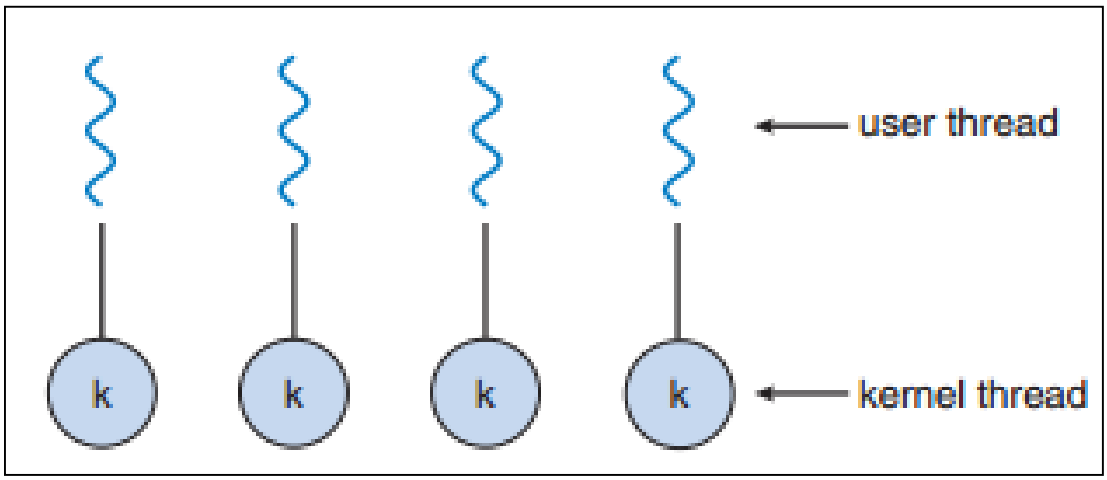


Hình 2-9: Đa luồng Many-To-One

Được quản lý bởi thư viện threads trong user space → hoạt động khá hiệu quả. Tuy nhiên, toàn bộ xử lý (process) sẽ bị khóa (block) nếu một thread tạo ra một ***blocking system call***.

Tại một thời điểm chỉ có một thread truy cập kernel. Nhiều threads không thể chạy song song trên ***multicore-systems*** → rất ít hệ thống sử dụng kiểu này.

7.2. One-To-One

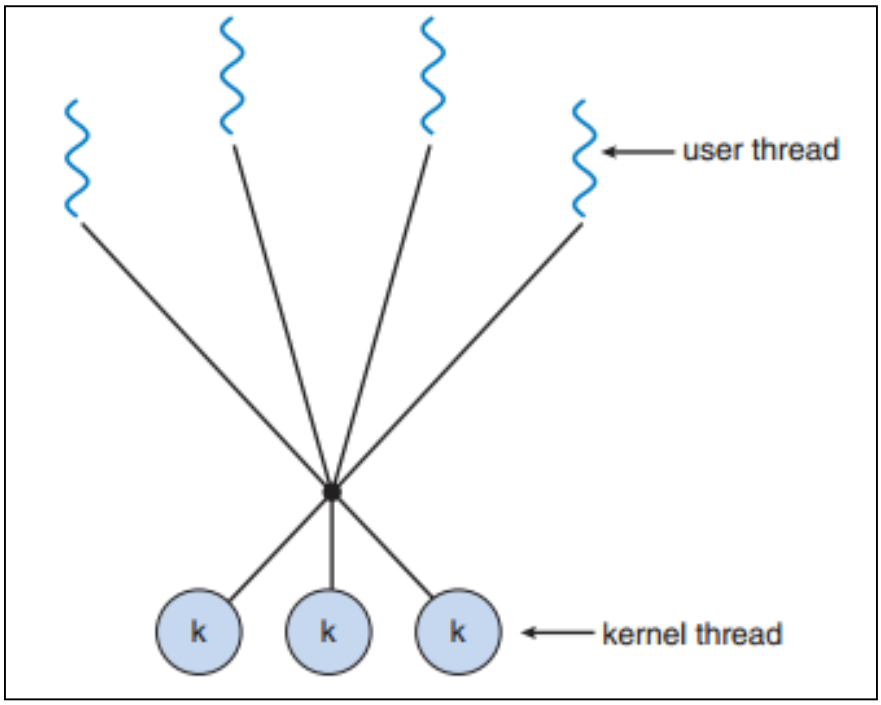


Hình 2-10: Đa luồng One-To-One

Cung cấp nhiều khả năng đồng thời hơn so với kiểu đa luồng Many-to-One. Khi có 1 thread tạo ra một ***blocking system call*** → hệ thống vẫn cho phép thread khác hoạt động. Mô hình này cho phép nhiều threads chạy song song trên ***multi-processors***.

Hạn chế của mô hình này là khi tạo ra một user thread thì phải tạo ra một kernel thread tương ứng.

7.3. Many-To-Many



Hình 2-11: Đa luồng Many-To-Many

Ta nhận thấy số lượng **user threads** >= số lượng **kernel threads**.

Các nhà phát triển phần mềm có thể tạo nhiều user threads theo nhu cầu và nhiều kernel threads tương ứng có thể chạy song song trên hệ thống m***ultiprocessor***.

Khi một thread thực hiện một ***blocking system call*** → kernel có thể lập lịch cho một thread khác thực thi.

*****

CÂU HỎI ÔN TẬP

Câu 1:

Một tiến trình là:

a. Một chương trình được thực thi
b. Một chương trình trong hệ thống máy tính
c. Một chương trình được load va bộ nhớ và thực thi
d. Một tiến trình có nhiều luồng

Câu 2:

Tên nào sau đây không phải là một thành phần của tiến trình?

a. Code
b. Stack
c. Current activity
d. Text

Câu 3:

Mỗi tiến trình đều có một khối quản lý tiến trình để quản lý nó. Tên của khối quản lý tiến trình là:

a. BCP
b. TCB
c. PCB
d. Tất cả đều sai

Câu 4:

Mỗi tiến trình có 5 trạng thái chuyển đổi qua lại với nhau; 5 trạng thái đó là:

a. Begin, Ready, Running, Waiting, Terminated
b. Ready, New, Running, Waiting, Terminated
c. New, Waiting, Running, Ready, End
d. Start, Ready, Running, Waiting, End

Câu 5:

Có một cơ chế cho phép các tiến trình trao đổi dữ liệu với nhau; Tên của cơ chế đó được viết tắc là:

a. ICP
b. IPC
c. PCI
d. PCB

*****

ĐÁP ÁN

Câu 1: c

Câu 2: a

Câu 3: c

Câu 4: b

Câu 5: b

*****

Chương 3: LẬP LỊCH TIẾN TRÌNH

1. Giới thiệu về lập lịch

Lập lịch là nền tảng của các HĐH đa nhiệm; Cần thiết phải lập lịch để phân phối tài nguyên hợp lý cho các tiến trình cùng có nhu cầu, tránh tranh chấp mà rơi vào deadlock. CPU là một tài nguyên mà mọi tiến trình đầu có nhu cầu được cấp phát để thực thi.

CPU scheduler là cơ chế chuyển đổi CPU giữa các xử lý nhằm đảm bảo tiến trình nào cũng được cấp CPU để hoạt động, phát huy khả năng đa nhiệm của hệ thống máy tính và giúp máy tính hoạt động hiệu quả hơn. CPU scheduler liên quan đến 4 trường hợp sau:

- Khi 1 xử lý chuyển trạng thái từ **running → waiting** (ví dụ: I/O request)
- Khi 1 xử lý chuyển trạng thái từ **running → ready** (ví dụ: interrupted)
- Khi 1 xử lý chuyển trạng thái từ **waiting → ready** (ví dụ: I/O complete)
- Khi 1 xử lý **kết thúc đột ngột → ready**

Nếu có nhiều tiến trình cùng có nhu cầu CPU, chúng sẽ đứng đợi trong một hàng đợi (Processes Ready Queue).

Khi CPU rỗi, HĐH sẽ chọn một tiến trình trong hàng đợi này để cấp phát CPU cho nó thực thi. Hiển nhiên việc chọn một tiến trình để cấp CPU phụ thuộc vào nhiều yếu tố. Mỗi thuật toán lập lịch (Scheduling Algorithms) đều dựa vào một vài yếu tố khác nhau; Tuy nhiên, mục đích lựa chọn tiến trình của thuật toán lập lịch nhằm:

- Cấp phát CPU tối ưu nhất
- Thời gian đáp ứng cho users nhanh nhất

Việc lập lịch có thể dựa trên một số tiêu chí sau:

- CPU utilization (Khả năng sử dụng CPU)
- Throughput (Thông lượng)
- Turnaround time (Thời gian xoay vòng)
- Waiting time (Thời gian đợi)
- Response time (Thời gian đáp ứng)

2. Một số thuật toán lập lịch

Hiện nay có nhiều thuật toán lập lịch tiến trình được cài đặt vào HĐH, mỗi thuật toán có một ưu điểm riêng (kể cả khuyết điểm) và thích hợp trong một số tình huống cụ thể. Không có thuật toán nào toàn diện.

Chương này đề cập 4 thuật toán đơn giản và thông dụng:

- First-Come, First-Served Scheduling (**FCFS**)
- Shortest-Job-First Scheduling (**SJF**)
- Priority Scheduling (**PS**)
- Round-Robin Scheduling (**RR**)

Trong các thuật toán đề cập sau đây, giả thiết rằng có nhiều tiến trình đứng trong hàng đợi để chờ tới lượt nhận CPU.

2.1. Thuật toán FCFS

Tiêu chí lập lịch: **Thứ tự đứng trước trong hàng đợi**.

Đặc điểm:

- Tiến trình nào **yêu cầu CPU trước** → được **cấp phát CPU trước**.
- Sử dụng **FIFO queue**
- **Thích hợp** cho các **hệ thống tuần tự** (**batch systems**)
- **Không thích hợp** cho các **hệ thống tương tác** (**interactive systems**)

Sau đây là một số ví dụ:

Ví dụ 1:

Cho 3 xử lý đang đứng trong hàng đợi cùng thời điểm **0** (là thời điểm bắt đầu xét; không quan tâm thời gian thực tế) theo thứ tự là: **P1**, **P2**, **P3**. Nhu cầu sử dụng CPU (**CPU Time**) được tính bằng đơn vị milliseconds (***ms***) và được cho như trong bảng sau:

Processes	CPU Time
P1	24
P2	3
P3	3

Yêu cầu:

a. Vẽ biểu đồ (Gantt Chart).

b. Tính thời gian đợi trung bình của các xử lý.

c. Tính thời gian hoạt động (bao gồm thời gian đợi và thời gian thực thi) trung bình của các xử lý.

Giải:

a. Vẽ biểu đồ.

Với thứ tự các xử lý là P1, P2, P3, ta có biểu đồ sau:

b. Tính thời gian đợi trung bình của các xử lý.

Dựa vào bảng số liệu (giả thiết) và biểu đồ trong câu a, ta có:

- P1: 0 ms
- P2: 24 ms
- P3: 27 ms

$\mathbf{T_{đtb}}$ = (0 + 24 + 27)/3 = 17 ms

c. Tính thời gian hoạt động trung bình của các xử lý.

Dựa vào công thức: **(Thời gian hoạt động trung bình) =**

(Thời gian đợi) + (Thời gian sử dụng CPU)

Ta có:

- P1: 0 + 24 = 24 ms
- P2: 24 + 3 = 27 ms
- P3: 27 + 3 = 30 ms

Vậy $T_{hđtb}$ = (24 + 27 + 30)/3 = 27 ms

* **Nhận xét**:

Trong ví dụ 1, nếu ta thay đổi thứ tự các tiến trình là P2, P3, P1 thì thời gian đợi trung bình $T_{đtb}$ sẽ giảm xuống rất nhiều ($T_{đtb}$ = (0 + 3 + 6)/3 = 3 ms).

(Sinh viên tự kiểm tra lại)

=====

Ví dụ 2:

Cho 3 xử lý được đưa vào hàng đợi (xét tại thời điểm **0**) theo thứ tự là: **P1**, **P2**, **P3**. Nhu cầu sử dụng CPU (**CPU time**) tính bằng milliseconds (**ms**) như bảng sau:

Processes	Arrival Time	Burst Time
P1	0	8
P2	3	4
P3	5	7

Yêu cầu:

a. Vẽ biểu đồ

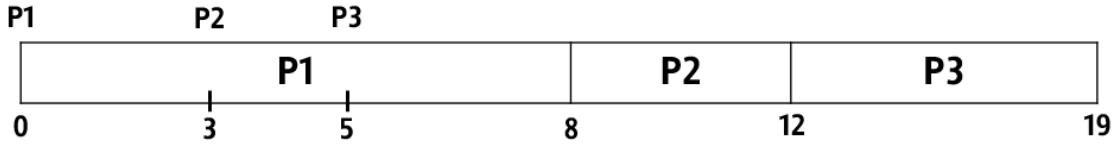
b. Tính **thời gian đợi trung bình** của các xử lý

c. Tính **thời gian hoạt động trung bình** của các xử lý

Giải:

Lưu ý trong ví dụ 2: Thời điểm bắt đầu xét (thời điểm 0), trong hàng đợi chỉ có P1, sau đó lần lượt P2 rồi P3 đến.

a. Vẽ biểu đồ.



b. Tính **thời gian đợi trung bình** của các xử lý

Dựa trên giả thiết và kết quả câu 1, ta có:

P1: 0 ms

P2: 8 – 3 = 5 ms

P3: 12 – 5 = 7 ms

$T_{đtb}$ = (0 + 5 + 7)/3 = 4 ms

c. Tính **thời gian hoạt động trung bình** của các xử lý

P1: 0 + 8 = 8 ms

P2: 5 + 4 = 9 ms

P3: 7 + 7 = 14 ms

$T_{hđtb}$ = (8 + 9 + 14)/3 = 31/3 = 10.33 ms

=====

2.2. Thuật toán SJF

Tiêu chí lập lịch: **dựa trên thời gian sử dụng CPU** (**CPU time**)

Đặc điểm:

- Cấp CPU cho tiến trình nào có **thời gian sử dụng CPU nhỏ nhất**
- **Nếu có 2 hoặc nhiều tiến trình có cùng thời gian sử dụng CPU nhỏ nhất → Thuật toán FCFS được sử dụng**

Ưu điểm: Cho thời gian đợi trung bình nhỏ nhất.

Khuyết điểm: Phải xác định thời gian sử dụng CPU kế tiếp.

Ví dụ 3:

Cho 4 tiến trình cùng xuất hiện trong hàng đợi (xét tại thời điểm **0**) với CPU time tương ứng như sau:

Processes	CPU Time
P1	6
P2	8
P3	7
P4	3

Yêu cầu:

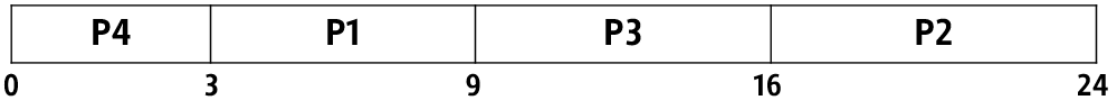
a. Vẽ biểu đồ.

b. Tính thời gian đợi trung bình của các xử lý.

c. Tính thời gian hoạt động trung bình của các xử lý.

Giải:

a. Vẽ biểu đồ.



b. Tính thời gian đợi trung bình của các xử lý.

P1: 3 ms

P2: 16 ms

P3: 9 ms

P4: 0 ms

$T_{đtb}$ = (3 + 16 + 9 + 0)/4 = **7** ms

c. Tính thời gian hoạt động trung bình của các xử lý.

P1: 3 + 6 = 9 ms

P2: 16 + 8 = 24 ms

P3: 9 + 7 = 16 ms

P4: 0 + 3 = 3 ms

$\mathbf{T_{hđtb}}$ = (9 + 24 + 16 + 3)/4 = **13** ms

=====

Ví dụ 4:

Cho 4 tiến trình: P1, P2, P3, P4 xuất hiện trong hàng đợi (xét tại thời điểm **0**) với CPU time và Arrival time (thời điểm tiến trình đứng vào hàng đợi) tương ứng như bảng sau:

Processes	Arrival Time	CPU Time
P1	0	8
P2	3	4
P3	5	7
P4	9	3

Yêu cầu:

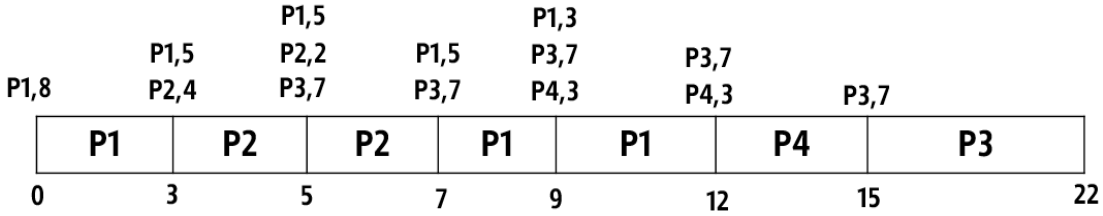
a. Vẽ biểu đồ

b. Tính thời gian đợi trung bình

c. Tính thời gian hoạt động trung bình

Giải:

a. Vẽ biểu đồ.



b. Tính thời gian đợi trung bình

P1: 7 – 3 = 4 ms

P2: 0 ms

P3: 15 – 5 = 10 ms

P4: 12 – 9 = 3 ms

$T_{đtb}$ = (4+0+10+3)/4 = 17/4 = 4.25 ms

c. Tính thời gian hoạt động trung bình

P1: 12 – 0 = 12 ms

P2: 7 – 3 = 4 ms

P3: 22 – 5 = 17 ms

P4: 15 – 9 = 6 ms

$T_{hđtb}$ = (12+4+17+6)/4 = 39/4 = 9.75 ms

=====

2.3. Thuật toán PS

Tiêu chí lập lịch: **dựa trên độ ưu tiên (priority) của tiến trình**

Trong HĐH, mỗi tiến trình đều có một giá trị về độ ưu tiên. Giá trị này do hệ thống gán hoặc do người quản trị hệ thống (admin) gán. Chúng là các giá trị nguyên bắt đầu từ 1. Giá trị càng nhỏ tương ứng với độ ưu tiên càng lớn; do đó giá trị ưu tiên 1 tương ứng với độ ưu tiên cao nhất.

Đặc điểm:

- Tiến trình đến sau có độ ưu tiên cao hơn → Sẽ được thực thi trước.
- Tiến trình đang thực thi nhưng có độ ưu tiên thấp → Sẽ phải tạm dừng.

Giả sử có 3 tiến trình:

- P1 có priority là 3
- P2 có priority là 1
- P3 có priority là 7

Khi đó: **P2** có priority cao nhất → thực thi đầu tiên.

P3 có priority thấp nhất → thực thi sau cùng.

Ví dụ 5:

Cho các tiến trình có thời gian đi vào hàng đợi (Arrival Time), thời gian sử dụng CPU (CPU Time) và độ ưu tiên (Priority) tương ứng như trong bảng sau:

Processes	Arrival Time	CPU Time	Priority
P1	0	5	2
P2	2	4	1
P3	3	7	3
P4	5	6	4

Yêu cầu:

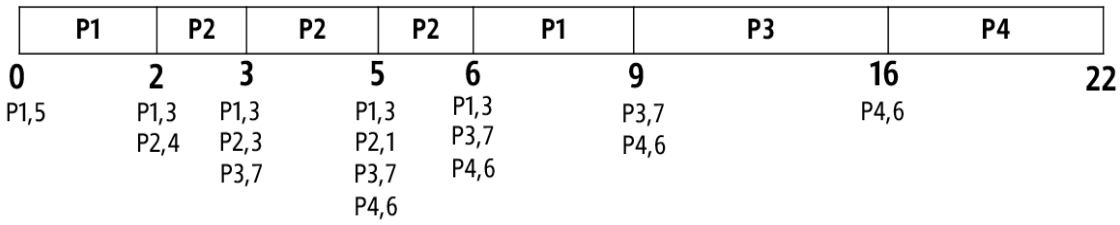
a. Vẽ **Gantt chart** biểu diễn sự thực thi của các xử lý.

b. Tính thời gian đợi trung bình của các xử lý.

c. Tính thời gian hoạt động trung bình của các xử lý

Giải:

a. Vẽ **Gantt chart** biểu diễn sự thực thi của các xử lý.



b. Tính thời gian đợi trung bình của các xử lý.

P1: 6 – 2 = 4 ms

P2: 0 ms

P3: 9 – 3 = 6 ms

P4: 16 – 5 = 11 ms

$T_{đtb}$ = (4+0+6+11)/4 = 21/4 = 5,25 ms

c. Tính thời gian hoạt động trung bình của các xử lý

P1: 4 + 5 = 9 ms

P2: 0 + 4 = 4 ms

P3: 6 + 7 = 13 ms

P4: 11 + 6 = 17 ms

$T_{hđtb}$ = (9+4+13+17)/4 = 43/4 = 10,75 ms

=====

Ví dụ 6:

Cho các tiến trình có thời gian đi vào hàng đợi (Arrival Time), thời gian sử dụng CPU (CPU Time) và độ ưu tiên (Priority) tương ứng như trong bảng sau:

Processes	Arrival Time	CPU Time	Priority
P1	0	3	3
P2	2	7	4
P3	3	5	1
P4	5	9	2

Yêu cầu:

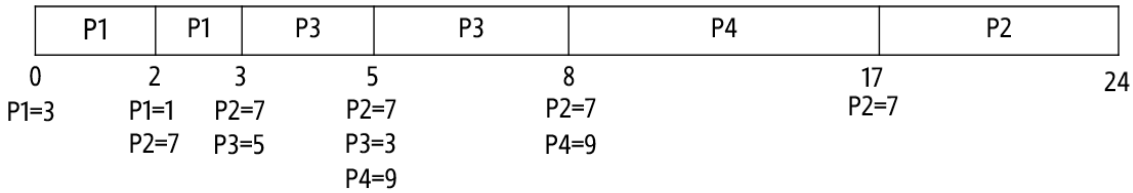
a. Vẽ **Gantt chart** biểu diễn sự thực thi của các xử lý.

b. Tính thời gian đợi trung bình của các xử lý.

c. Tính thời gian hoạt động trung bình của các xử lý

Giải:

a. Vẽ **Gantt chart** biểu diễn sự thực thi của các xử lý.



b. Tính thời gian đợi trung bình của các xử lý.

P1: 0 ms

P2: 17 – 2 = 15 ms

P3: 0 ms

P4: 8 – 5 = 3 ms

$T_{đtb}$ = (0 + 15 + 0 + 3)/4 = 18/4 = 4,5 ms

c. Tính thời gian hoạt động trung bình của các xử lý

P1: 0 + 3 = 3 ms

P2: 15 + 7 = 22 ms

P3: 0 + 5 = 5 ms

P4: 3 + 9 = 12 ms

$T_{hđtb}$ = (3 + 22 + 5 + 12)/4 = 42/4 = 10,5 ms

2.4. Thuật toán RR

Thuật toán này thường được sử dụng trong các hệ thống chia sẻ thời gian thực (time-sharing systems)

Tiêu chí lập lịch: **dựa trên vị trí đứng trước trong hàng đợi và thời gian xoay vòng (Time Slide) của tiến trình**

Time Slide (hay Time Quantum) có giá trị từ 10 đến 100 ms. Tuy nhiên, để dể tính toán và theo dõi, trong các ví dụ của phần này, chúng ta sử dụng time slide với các giá trị nhỏ hơn 10.

Đặc điểm:

- Thuật toán cấp phát thời gian sử dụng CPU cho mỗi tiến trình dựa trên **time quantum**.
- Một tiến trình nếu có thời gian thực thi (còn lại) nhỏ hơn time quantum → sau khi thực thi xong → process đó được giải phóng và CPU được cấp cho tiến trình kế tiếp.
- Một tiến trình nếu có thời gian thực thi lớn hơn time quantum → sau khi thực thi hết một time quantum → process đó được đưa vào cuối **ready queue** (đợi đến lượt cấp CPU kế tiếp).

Ví dụ 7:

Cho các tiến trình đều đứng trong hàng đợi tại thời điểm 0 và giá trị **time quantum là 4 ms**, như trong bảng sau:

Processes	CPU Time
P1	24
P2	3
P3	3

Yêu cầu:

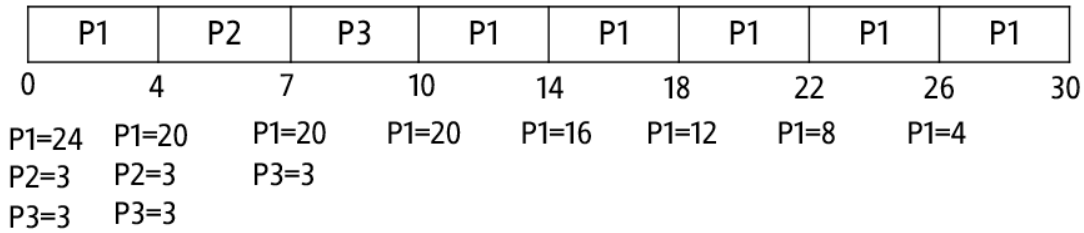
a. Vẽ **Gantt chart** biểu diễn sự thực thi của các xử lý.

b. Tính thời gian đợi trung bình của các xử lý.

c. Tính thời gian hoạt động trung bình của các xử lý

Giải:

a. Vẽ **Gantt chart** biểu diễn sự thực thi của các xử lý.



b. Tính thời gian đợi trung bình của các xử lý.

P1: 10-4=6 ms

P2: 4-0=4 ms

P3: 7-0=7 ms

$T_{đtb}$ = (6 + 4 + 7)/3 ≈ 5.67 ms

c. Tính thời gian hoạt động trung bình của các xử lý

P1: 6 + 24 = 30 ms

P2: 4 + 3 = 7 ms

P3: 7 + 3 = 10 ms

$T_{hđtb}$ = (30 + 7 + 10)/3 = 47/3 ≈ 15.67 ms

=====

Ví dụ 8:

Cho các tiến trình lần lượt đứng vào hàng đợi (xét tại thời điểm 0) và giá trị **time quantum là 3 ms**, như trong bảng sau:

Processes	Arrival Time	CPU Time
P1	0	6
P2	1	3
P3	2	1
P4	3	4

Yêu cầu:

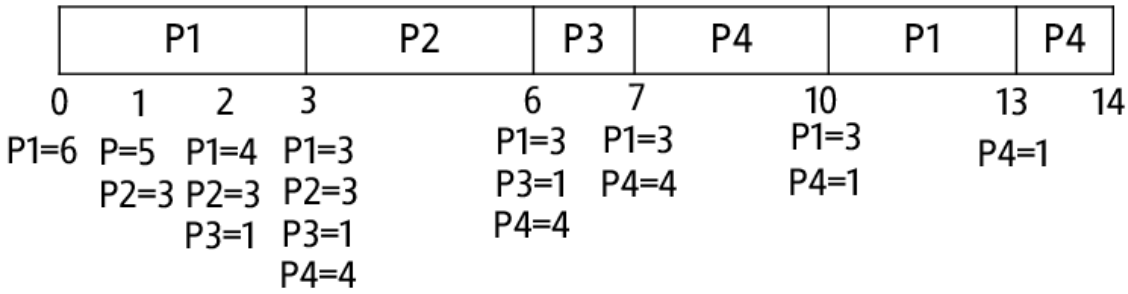
a. Vẽ **Gantt chart** biểu diễn sự thực thi của các xử lý.

b. Tính thời gian đợi trung bình của các xử lý.

c. Tính thời gian hoạt động trung bình của các xử lý

Giải:

a. Vẽ **Gantt chart** biểu diễn sự thực thi của các xử lý.



b. Tính thời gian đợi trung bình của các xử lý.

P1: 10 – 3 = 7 ms

P2: 3 – 1 = 2 ms

P3: 6 – 2 = 4 ms

P4: (7 - 3) + (13 - 10) = 7 ms

$T_{đtb}$ = (7 + 2 + 4 + 7)/4 = 5 ms

c. Tính thời gian hoạt động trung bình của các xử lý

P1: 7 + 6 = 13 ms

P2: 2 + 3 = 5 ms

P3: 4 + 1 = 5 ms

P4: 7 + 4 = 11 ms

$T_{hđtb}$ = (13 + 5 + 5 + 11)/4 = 34/4 = 8.5 ms

=====

CÂU HỎI ÔN TẬP

Câu 1:

Mục đích lựa chọn tiến trình để cấp CPU của thuật toán lập lịch nhằm:

a. Cấp phát CPU tối ưu nhất
b. Một chương trình trong hệ thống máy tính
c. Thời gian đáp ứng cho users nhanh nhất
d. a và c đúng

Câu 2:

Với thuật toán lập lịch FCFS, tiêu chí chọn lựa tiến trình để cấp CPU là:

a. Độ ưu tiên của tiến trình
b. Ví trí trong hàng đợi của tiến trình
c. Thời gian sử dụng CPU của tiến trình bé nhất
d. Hệ thống chọn ngẫu nhiên vì nó cho rằng như vậy là hợp lý.

Câu 3:

Với thuật toán lập lịch SJF, tiêu chí chọn lựa tiến trình để cấp CPU là:

a. Độ ưu tiên của tiến trình
b. Ví trí trong hàng đợi của tiến trình
c. Thời gian sử dụng CPU của tiến trình bé nhất
d. Sự quan trọng của tiến trình đối với hệ thống.

Câu 4:

Với thuật toán lập lịch PS, tiêu chí chọn lựa tiến trình để cấp CPU là:

a. Độ ưu tiên của tiến trình
b. Ví trí trong hàng đợi của tiến trình
c. Thời gian sử dụng CPU của tiến trình bé nhất
d. Vị trí đứng trước trong hàng đợi và thời gian sử dụng CPU bé nhất

Câu 5:

Với thuật toán lập lịch RR, tiêu chí chọn lựa tiến trình để cấp CPU là:

a. Độ ưu tiên của tiến trình
b. Ví trí trong hàng đợi của tiến trình

c. Thời gian sử dụng CPU của tiến trình bé nhất

d. Kết hợp FCFS với Time Quantum

====

ĐÁP ÁN

Câu 1: d

Câu 2: b

Câu 3: c

Câu 4: a

Câu 5: d

=====

BÀI TẬP

Bài tập 1:

Cho 3 xử lý được đưa vào hàng đợi, xét tại thời điểm 0 theo thứ tự là: P1, P2, P3. Nhu cầu sử dụng CPU (CPU time) tính bằng milliseconds (ms) như bảng sau:

Processes	Arrival Time	CPU Time
P1	0	8
P2	3	4
P3	5	7

Yêu cầu:

a. **Vẽ biểu đồ**

b. Tính **thời gian đợi trung bình** của các xử lý

c. Tính **thời gian hoạt động trung bình** của các xử lý

***

Bài tập 2:

Xét 4 xử lý được đưa vào hàng đợi bắt đầu tại thời điểm **0** theo thứ tự là: **P1**, **P2**, **P3, P4** như hình:

Processes	Arrival Time	CPU Time
P1	2	8
P2	4	4
P3	7	7
P4	9	3

Yêu cầu:

a. **Vẽ biểu đồ**

b. Tính **thời gian đợi trung bình** của các xử lý

c. Tính **thời gian hoạt động trung bình** của các xử lý

***

Bài tập 3:

Cho 4 xử lý được đưa vào hàng đợi tại thời điểm 0 theo thứ tự là: P1, P2, P3, P4 như bảng:

Processes	Arrival Time	CPU Time
P1	3	8
P2	5	6
P3	7	7
P4	10	1

Yêu cầu:

a. **Vẽ biểu đồ**

b. Tính **thời gian đợi trung bình** của các xử lý

c. Tính **thời gian hoạt động trung bình** của các xử lý

***

Bài tập 4:

Xét các xử lý đến hàng đợi tại các thời điểm khác nhau, như bảng sau:

Processes	Arrival Time	CPU Time
P1	0	8
P2	2	4
P3	4	9
P4	6	5

Yêu cầu:

a. **Vẽ biểu đồ**

b. Tính **thời gian đợi trung bình** của các xử lý

c. Tính **thời gian hoạt động trung bình** của các xử lý

***

Bài tập 5:

Xét các xử lý có độ ưu tiên đi đến hàng đợi tại các thời điểm khác nhau, như bảng sau:

Processes	Arrival Time	CPU Time	Priority
P1	2	5	3
P2	3	7	1
P3	5	5	4
P4	6	6	2

Yêu cầu:

a. **Vẽ biểu đồ**

b. Tính **thời gian đợi trung bình** của các xử lý

c. Tính **thời gian hoạt động trung bình** của các xử lý

***

Bài tập 6:

Xét các xử lý có độ lần lượt đứng vào hàng đợi tại các thời điểm khác nhau, như bảng sau:

Processes	Arrival Time	CPU Time
P1	1	4
P2	3	7
P3	4	6
P4	6	5

Cho time quantum = 3 ms.

Yêu cầu:

a. **Vẽ biểu đồ**

b. Tính **thời gian đợi trung bình** của các xử lý

c. Tính **thời gian hoạt động trung bình** của các xử lý

***

Chương 4: DEADLOCK

1. Giới thiệu về deadlock

Trong môi trường đa nhiệm, thường xuất hiện tình huống nhiều tiến trình cạnh tranh tài nguyên. Nếu tài nguyên không có sẵn thì tiến trình sẽ chuyển qua trạng thái đợi (waiting).

Nếu đợi được tài nguyên → livelock.

Nếu đợi không được tài nguyên → deadlock.

Các kiểu tài nguyên của hệ thống:

- CPU
- Memory
- IO devices
- **…**

Các hành động liên quan đến tài nguyên:

- request (yêu cầu)
- use (sử dụng)
- release (giải phóng)

Deadlock sẽ xuất hiện nếu 4 trường hợp sau **đồng thời xảy ra** trong hệ thống:

- **Mutual exclusion**: chỉ 1 tiến trình có thể sử dụng resource tại 1 thời điểm.
- **Hold and wait**: 1 tiến trình **đang đợi** và **đang giữ** ít nhất 1 tài nguyên mà các tiến trình khác đang cần.
- **No preemption**: 1 tài nguyên chỉ được giải phóng khi tiến trình đang chiếm giữ nó kết thúc công việc.
- **Circular wait:**

  Có **n** tiến trình đang đợi tài nguyên là: P1, P2, …, Pn

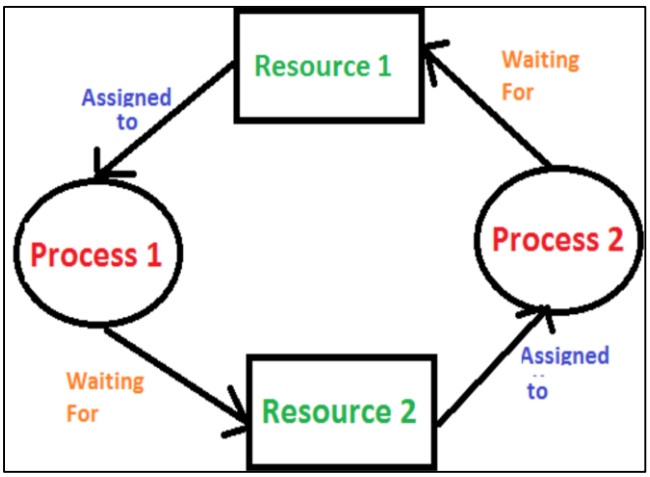
  P1 đang đợi tài nguyên mà **P2** đang giữ,

  P2 đang đợi tài nguyên mà **P3** đang giữ,

  …

  Pn đang đợi tài nguyên mà **P1** đang giữ.

  → **Tạo thành chu trình đợi tài nguyên** (như hình 4-1).



Hình 4-1: Chu trình đợi tài nguyên

2. Các phương thức quản lý deadlock

Để đối phó với deadlock, cần phải sử dụng 1 trong 3 cách sau:

Cách 1: Dùng giao thức để ngăn chặn hay tránh deadlock, đảm bảo rằng hệ thống không bao giờ rơi vào trạng thái deadlock.

Cách 2: Cho phép hệ thống deadlock, sau đó phát hiện nó và phục hồi nó.

Cách 3: Bỏ qua vấn đề deadlock và xem như deadlock không bao giờ xảy ra.

3. Ngăn chặn deadlock

Từ mục 1 trong chương này, ta biết deadlock xuất hiện nếu như có 4 điều kiện đồng thời xảy ra. Do đó, để tránh deadlock, cần phải đảm bảo ít nhất 1 trong 4 trường hợp deadlock không thể xảy ra. Tuy nhiên điều này có thể ảnh hưởng đến hiệu quả của hệ thống.

Chúng ta lần lượt xem xét từng trường hợp:

3.1. Mutual exclusion

Các **tài nguyên có thể share**, không bắt buộc phải loại trừ lẫn nhau (mutual exclusion); **Ví dụ**: **Read-only files** có thể cho phép **nhiều truy cập cùng lúc**. Trong thực tế không thể cấm triệt để trường hợp mutual exclusion.

3.2. Hold and wait

Đảm bảo điều kiện **hold-and-wait** không bao giờ xuất hiện trong hệ thống. Cụ thể:

- Đảm bảo 1 tiến trình khi yêu cầu 1 tài nguyên thì không thể đang nắm giữ các tài nguyên khác.
- Cấp đầy đủ tài nguyên cho 1 tiến trình trước khi nó bắt đầu thực thi. Nếu không thể cấp đủ tài nguyên thì không cấp.

3.3. No preemption

Nếu 1 tiến trình đang nắm giữ một số tài nguyên và đang cần một số tài nguyên khác nhưng không có sẵn (nó phải đợi) thì tất cả tài nguyên nó đang giữ sẽ được giải phóng (cho các tiến trình khác).

Tất cả tài nguyên mà tiến trình đã giải phóng và tất cả tài nguyên nó đang đợi sẽ được đưa vào hàng đợi tài nguyên của nó.

Tiến trình sẽ được thực thi khi tất cả tài nguyên mà nó cần đã được cấp phát đầy đủ.

3.4. Circular wait

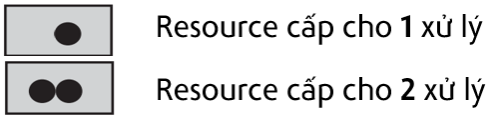
Áp đặt một số thứ tự cho mỗi tài nguyên và bắt buộc mỗi tiến trình yêu cầu tài nguyên phải tuân theo thứ tự tăng dần → Nhằm tránh tạo chu trình đợi tài nguyên.

Ví dụ: Một tiến trình đang giữ tài nguyên được gán số 5 thì nó chỉ có thể yêu cầu tài nguyên được gán số từ 6 trở lên; không được dưới 5.

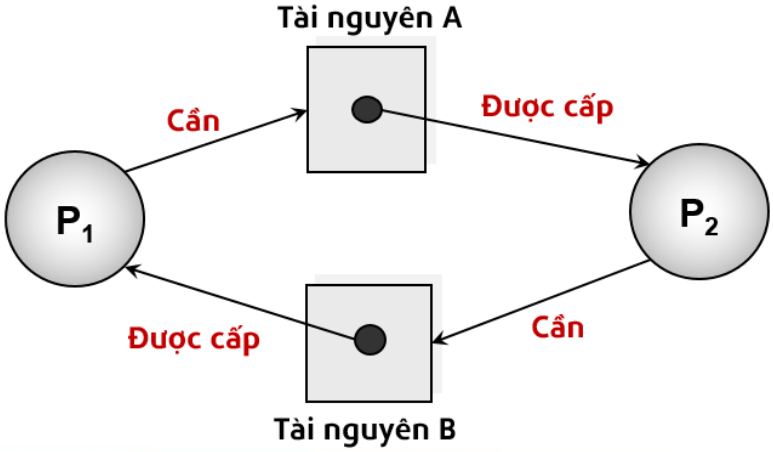
Một công cụ hay giúp việc cấp phát và theo dõi tài nguyên là **biểu đồ cấp phát tài nguyên** (Resource Allocation Graph-**RAG**).

Xem ký hiệu và biểu đồ ví dụ sau:

Ký hiệu:



Biểu đồ:



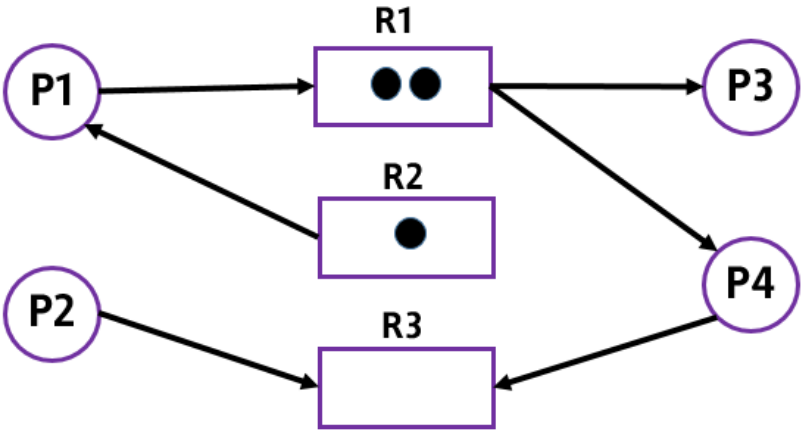
Hình 4-2: Biểu đồ cấp phát tài nguyên

Trong thực tế, khi có nhiều yêu cầu cấp phát tài nguyên, nên sử dụng biểu đồ cấp phát tài nguyên nhằm tránh tạo chu trình. Sau đây là một ví dụ:

Cho trạng thái các tiến trình như sau:

P1→R1, P2→R3, R2→P1, R1→P3, P4→R3, R1→P4

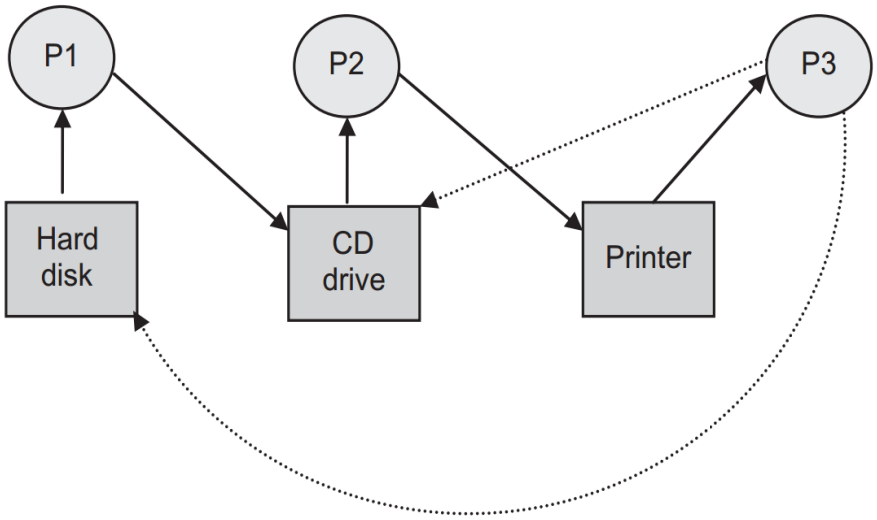
Yêu cầu: Vẽ **RAG.**



Hình 4-3: Một ví dụ biểu đồ cấp phát tài nguyên

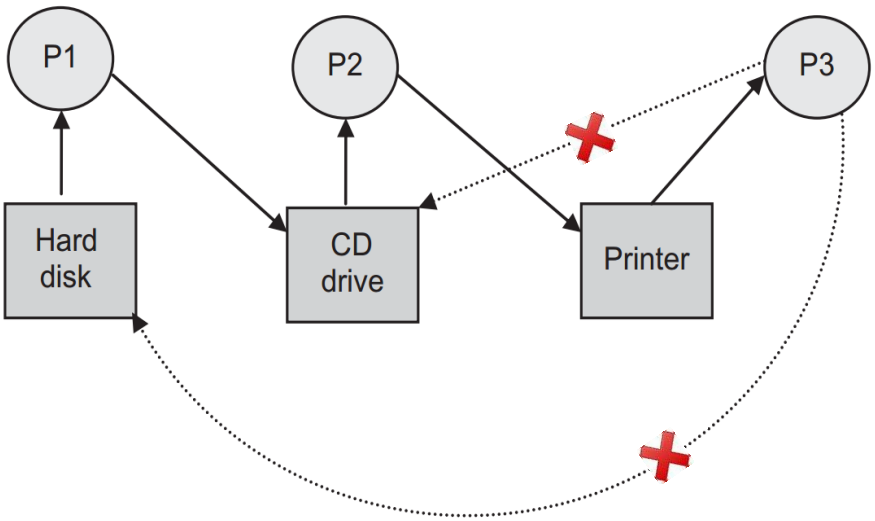
Nhận xét: Biểu đồ không xuất hiện chu trình → Các tiến trình không đợi chờ tài nguyên lẫn nhau → Tránh được deadlock.

Một ví dụ khác về biểu đồ RAG có **xuất hiện chu trình**:



Hình 4-4: Một ví dụ biểu đồ RAG xuất hiện chu trình

Giải pháp khắc phục: Tìm cách loại bỏ các đường cấp phát (mũi tên) có khả năng tạo chu trình. Trong hình vẽ 4-5 ta bỏ đi 2 đường chấm chấm.



Hình 4-5: Loại bỏ 2 đường cấp phát tài nguyên để tránh chu trình

3.5. Tránh deadlock

Khuyết điểm của việc ngăn chặn deadlock là làm giảm hoạt động của hệ thống. Một phương pháp khác để tránh deadlock là cung cấp tài nguyên bổ sung được yêu cầu.

Giải thuật tránh deadlock bao gồm 2 đặc điểm:

- Dựa trên trạng thái cấp phát tài nguyên
- Đảm bảo không tồn tại chu trình trong đồ thị cấp phát tài nguyên.

Xét trường hợp sau:

- Xử lý P giữ tài nguyên A rồi cần tiếp tài nguyên B.
- Xử lý Q giữ tài nguyên B rồi cần tài nguyên A.
- Hệ thống sẽ kiểm tra tài nguyên yêu cầu của P và Q để quyết định một xử lý có phải đợi hay không?

Có 2 trường hợp xảy ra:

Trường hợp 1: Yêu cầu tài nguyên của xử lý có khả năng dẫn đến deadlock → yêu cầu này phải đợi (unsafe).

Trường hợp 2: Yêu cầu tài nguyên của xử lý không dẫn đến deadlock → yêu cầu này được chấp nhận (safe).

3.6. Phát hiện deadlock

Nếu một hệ thống không thể ngăn chặn deadlock hay không thể tránh deadlock thì khi deadlock xảy ra, hệ thống phải cung cấp các giải pháp sau:

- Giải thuật xem xét trạng thái của hệ thống để quyết định deadlock có xảy ra hay không.
- Xác định xử lý nào gây ra deadlock → Reset xử lý đó.
- Giải thuật phục hồi từ deadlock: phục hồi công việc của xử lý bị reset.

Khi nào cần phải phát hiện deadlock?

Câu trả lời phụ thuộc vào hai yếu tố sau:

- Deadlock có khả năng xảy ra thường xuyên hay không?

  Nếu thường xuyên → Cần có cơ chế phát hiện.

  Nếu không thường xuyên → Có thể không quan tâm
- Bao nhiêu tiến trình sẽ bị ảnh hưởng bởi deadlock khi nó xảy ra?

  Mục đích của câu trả lời này là để chọn lựa giải thuật hay cách thức khôi phục hệ thống sau deadlock.

3.7. Phục hồi sau deadlock

Có 2 cách phục hồi:

- Thông báo người quản trị rằng deadlock xảy ra và để người quản trị giải quyết deadlock thủ công.
- Để hệ thống tự động phục hồi.

Có 2 tùy chọn dùng để phá vỡ deadlock:

- Huỷ bỏ một hay nhiều tiến trình để phá vỡ việc tồn tại chu trình trong đồ thị cấp phát tài nguyên.
- Lấy lại một số tài nguyên từ một hay nhiều tiến trình có khả năng gây ra deadlock.

Các yếu tố sau đây có thể sử dụng để xác định tiến trình nào cần bị hủy bỏ để tránh deadlock:

- Độ ưu tiên của tiến trình.
- Thời gian đã hoạt động và thời gian sắp kết thúc.
- Bao nhiêu tài nguyên và loại tài nguyên gì tiến trình đang sử dụng.
- Bao nhiêu tài nguyên cần cung cấp tiếp để tiến trình hoàn thành.
- Tiến trình là tương tác hay dạng bó (batch).

Việc hủy bỏ 1 tiến trình sẽ ảnh hưởng đến công việc do tiến trình đó thực hiện, nhất là những tiến trình đang hoạt động giữa chừng. Cần có giải pháp để phục hồi (reset) các giá trị mà tiến trình đó đã thay đổi hay bị hủy bỏ.

Ví dụ như với máy rút tiền tự động (ATM); Sau khi đã phát thông báo trừ tiền (tiền đã trừ trong tài khoản khách hàng) nhưng vì lý do gì đó mà không rút được tiền thì hệ thống phải hoàn tiền lại cho khách hàng.

*****

CÂU HỎI ÔN TẬP

Câu 1:

Hành động nào sau đây không liên quan đến việc sử dụng tài nguyên?

a. Stop
b. Request
c. Use
d. Release

Câu 2:

Điều kiện nào sau đây không ảnh hưởng đến việc deadlock xuất hiện?

a. Mutual exclusion
b. Stop and wait
c. Hold and wait
d. Circular wait

Câu 3:

Cách nào sau đây không được sử dụng để đối phó với deadlock?

a. Dùng 1 giao thức để ngăn chặn hay tránh deadlocks, đảm bảo rằng hệ thống không bao giờ rơi vào trạng thái deadlock.
b. Cho phép hệ thống vào trạng thái deadlock, sau đó phát hiện nó và phục hồi nó.
c. Bỏ qua vấn đề deadlock và giả vờ rằng deadlock không bao giờ xảy ra.
d. Loại bỏ deadock ngay từ đầu.

Câu 4:

Hành động nào sau đây không thể áp dụng khi sử dụng điều kiện Hold and wait để ngăn chặn deadlock?

a. Đảm bảo điều kiện hold-and-wait không bao giờ xuất hiện trong hệ thống.
b. Đảm bảo 1 xử lý khi yêu cầu 1 tài nguyên thì không thể đang nắm giữ các tài nguyên khác.
c. Chỉ cho phép 1 xử lý thực thi trong hệ thống.
d. Cấp đầy đủ tài nguyên cho 1 xử lý trước khi nó bắt đầu thực thi.

Câu 5:

Yếu tố nào sau đây không thể sử dụng để xác định tiến trình cần bị hủy bỏ khi bị deadlock?

a. Độ ưu tiên của tiến trình.
b. Kích thước của tiến trình.
c. Thời gian đã hoạt động và thời gian sắp kết thúc.
d. Tiến trình thuộc dạng giao tiếp hay dạng bó.

*****

ĐÁP ÁN

Câu 1: a

Câu 2: b

Câu 3: d

Câu 4: c

Câu 5: b

=====

Chương 5: QUẢN LÝ BỘ NHỚ

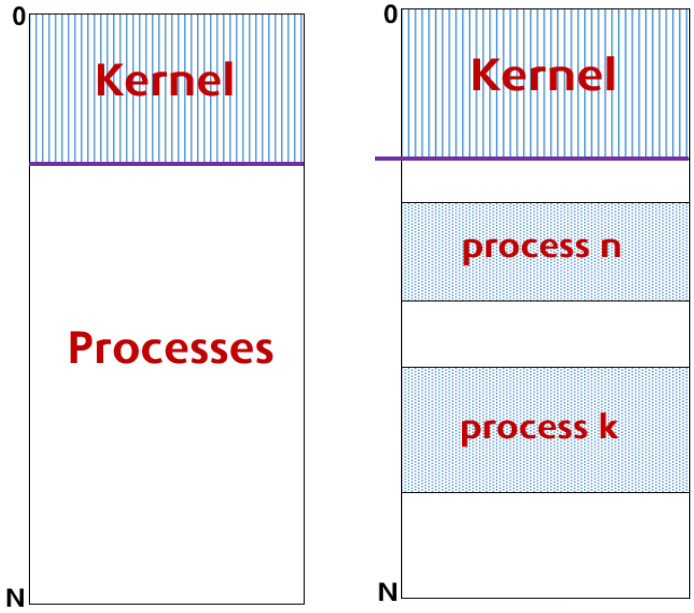
1. Một số khái niệm

Quản lý bộ nhớ là công việc của HĐH, kết hợp với sự hỗ trợ của phần cứng. Khi một tiến trình được phép thực thi, nó yêu cầu được cấp phát bộ nhớ. Có rất nhiều tiến trình đang hoạt động trong hệ thống, do đó cần phải có cơ chế phân phối và quản lý bộ nhớ hiệu quả, đảm bảo tiết kiệm bộ nhớ, tạo điều kiện cho càng nhiều tiến trình cùng hoạt động.

Mục tiêu hệ thống là nạp càng nhiều tiến trình vào bộ nhớ càng tốt, nhằm gia tăng mức độ đa nhiệm.

Trong hầu hết các hệ thống:

- Kernel chiếm 1 phần của bộ nhớ.
- Phần còn lại phân phối cho các tiến trình.



Hình 5-1: Mô phỏng việc phân chia bộ nhớ

Chức năng của quản lý bộ nhớ (Memory Management):

- Cấp phát bộ nhớ cho các tiến trình
- Cấp lại bộ nhớ khi cần thiết
- Kiểm tra việc truy xuất bộ nhớ hợp lệ
- Cho phép các tiến trình chia sẻ vùng nhớ chung
- Kết gán địa chỉ bộ nhớ luận lý của user với địa chỉ bộ nhớ vật lý của hệ thống

Trong chương này ta chỉ đề cập đến bộ nhớ máy tính mặc định là RAM; Cho dù máy tính có nhiều loại bộ nhớ. Khi nào đề cập đến loại bộ nhớ khác thì sẽ nói rõ.

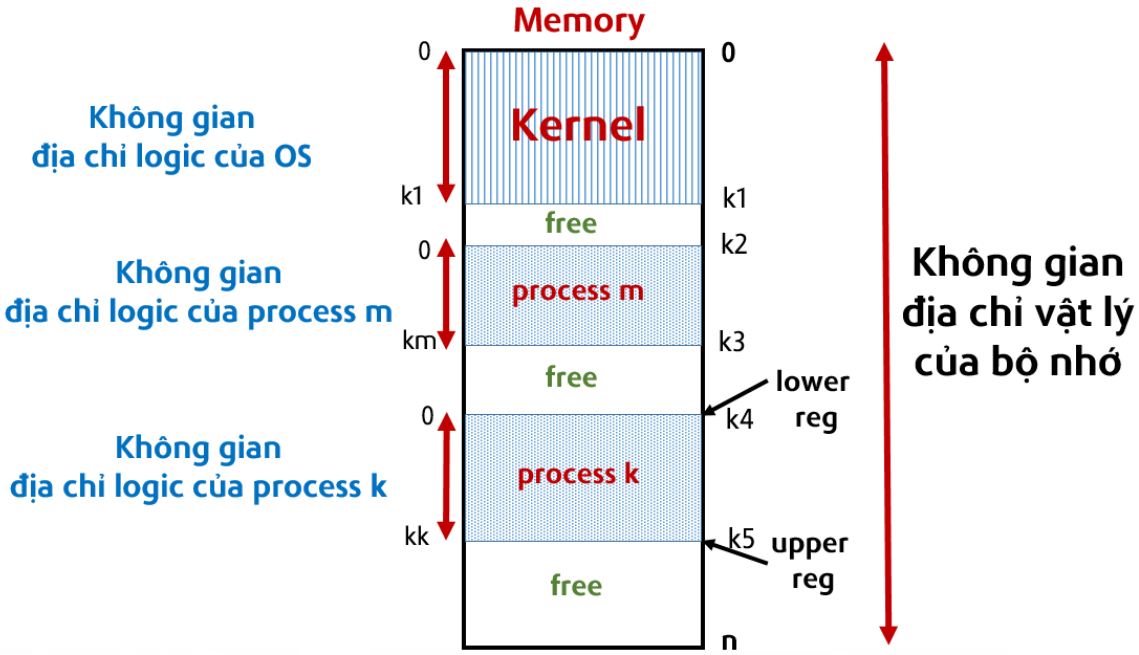
Bộ nhớ vật lý của máy tính (RAM) được mô phỏng như một hình chữ nhật (đứng/ngang), bao gồm nhiều ô nhớ, mỗi ô nhớ có kích thước 1 byte và được gắn một giá trị nguyên, bắt đầu từ 0, gọi là địa chỉ của ô nhớ. Máy tính có lượng RAM càng lớn thì số lượng ô nhớ càng nhiều.

Mỗi tiến trình có một không gian địa chỉ riêng; Đó là vùng nhớ mà hệ thống cấp cho tiến trình hoạt động, bao gồm nhiều ô nhớ liên tiếp nhau. Do đó, cần phải xác định phạm vi địa chỉ hợp lệ mà tiến trình có thể truy cập (đọc, ghi), tránh truy cập ra ngoài phạm vi vùng nhờ được cấp.

Việc xác định không gian địa chỉ của tiến trình dựa trên 2 thanh ghi:

- Base register (BR) (Lower-bound Register)
- Limit register (LR) (Upper-bound Register)

Ví dụ:



Hình 5-2: Mô phỏng không gian địa chỉ vùng nhớ cấp cho các tiến trình

Trong hệ thống có 2 phương thức cấp pháp bộ nhớ:

- **Tĩnh** (static allocation): Được thực hiện trước khi xử lý thực thi (compile time, load time)
- **Động** (dynamic allocation): Được thực hiện sau khi xử lý bắt đầu thực thi (run time)

Chi tiết 2 loại cấp phát bộ nhớ sẽ được đề cập ở phần sau.

2. Các kiểu địa chỉ

2.1. Địa chỉ vật lý (Physical Address)

Địa chỉ vật lý của ô nhớ tham chiếu đến vị trí của ô nhớ trong bộ nhớ máy tính (RAM). Nó là giá trị số mà ô nhớ được gắn trước khi bộ nhớ được tạo và đưa vào sử dụng. Địa chỉ này không thay đổi sau khi được hình thành (cho dù máy tính shutdown).

Mỗi ô nhớ đều có một địa chỉ vậy lý (duy nhất).

Địa chỉ vật lý của một vùng nhớ (bao gồm nhiều ô nhớ vật lý liên tiếp) là địa chỉ vật lý của ô nhớ đầu tiên (nhỏ nhất) thuộc vùng nhớ đó.

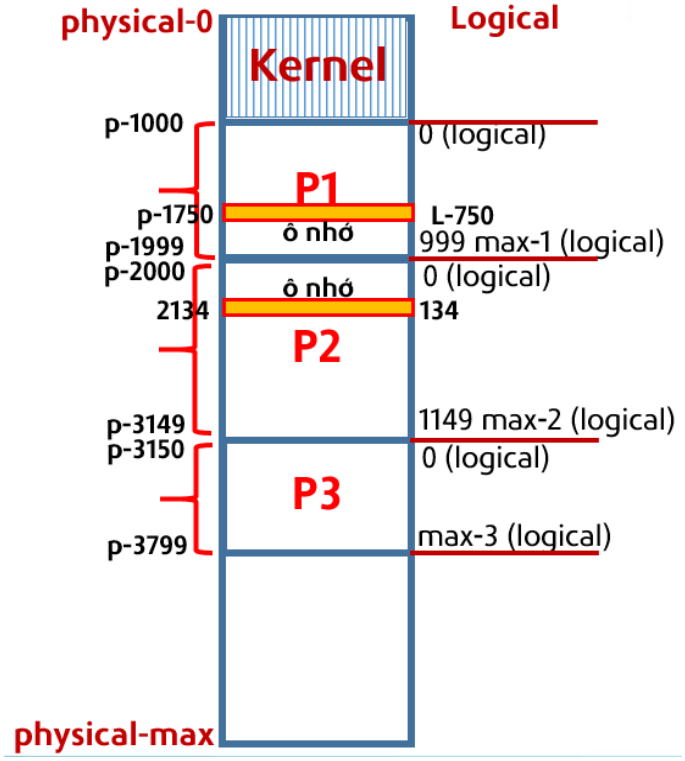
2.2. Địa chỉ luận lý (Logical Address)

Khi một vùng nhớ được cấp cho một tiến trình để hoạt động, tiến trình đó sẽ quản lý (toàn quyền) trên vùng nhớ này; Do đó, tiến trình sẽ gán lại địa chỉ cho các ô nhớ nằm trong vùng nhớ mà nó quản lý.

Mỗi ô nhớ (trong vùng nhớ được cấp) sẽ được gán một giá trị, là một số nguyên bắt đầu từ 0, gọi là địa chỉ luận lý. Khi đó, mỗi ô nhớ sẽ có 2 loại địa chỉ là: địa chỉ vật lý và địa chỉ luận lý.

Khi tiến trình kết thúc, vùng nhớ được giải phóng, các địa chỉ luận lý không còn nữa nhưng địa chỉ vật lý của ô nhớ vẫn giữ nguyên.

Hình vẽ sau đây cho thấy việc gán địa chỉ vật lý, địa chỉ luận lý:



Hình 5-3: Mô phỏng việc gán địa chỉ vật lý và luận lý.

2.3. Chuyển đổi giữa địa chỉ vật lý và địa chỉ luận lý

Ta có 2 công thức chuyển đổi sau:

Công thức 1: Chuyển đổi từ Logical address → Physical address

Logical address + Base Register = Physical Address

Công thức 2: Chuyển đổi từ Physical address → Logical address:

Physical address - Base Register = Logical Address

* **Lưu ý**: Kết quả chuyển đổi phải đối chiếu với không gian nhớ để đảm bảo các địa chỉ nhớ hợp lệ.

Ví dụ 1:

Cho xử lý P1 có base register là 1000 và limit register là 2500. Xử lý P2 có base register là 3000 và limit register là 4000.

P1 có lệnh A1 tại logical address là 500 và lệnh B1 tại logical address là 700, P2 có lệnh A2 tại logical address là 250 và lệnh B2 tại logical address là 600.

Yêu cầu:

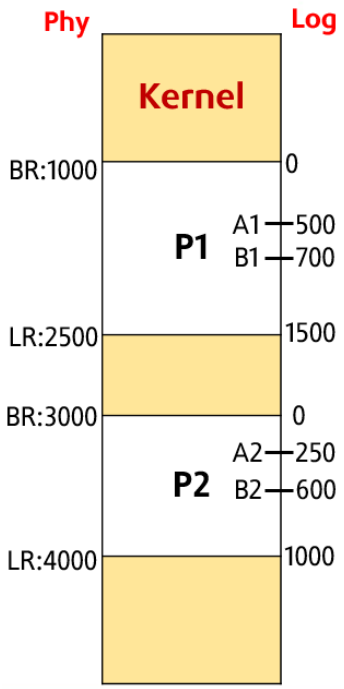
a. Xác định không gian logical address của xử lý P1 và P2.

b. Xác định physical address của các lệnh: A1, B1 của P1

c. Xác định physical address của các lệnh: A2, B2 của P2

Giải:

Dựa trên giả thiết ta vẽ phát họa mô hình chứa không gian địa chỉ của P1, P2 và các lệnh A1, B1, A2, B2.



a. Xác định không gian logical address của xử lý P1 và P2.

Để tính không gian địa chỉ luận lý của P1, ta tính địa chỉ luận lý của ô nhớ đầu tiên của P1 theo công thức 2:

1000 – 1000 = 0

Tiếp tục tính địa chỉ luận lý của ô nhớ cuối cùng của P1 theo công thức 2:

2500 – 1000 = 1500

Vậy không gian địa chỉ luận lý của P1 là: 0 – 1500.

(đối chiếu hình vẽ)

Để tính không gian địa chỉ luận lý của P2, ta tính địa chỉ luận lý của ô nhớ đầu tiên của P2 theo công thức 2:

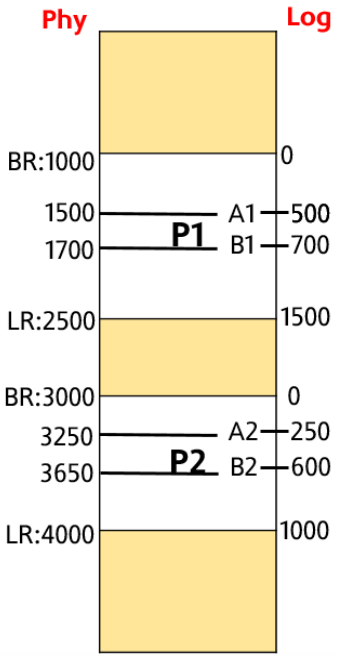
3000 – 3000 = 0

Tiếp tục tính địa chỉ luận lý của ô nhớ cuối cùng của P2 theo công thức 2:

4000 – 3000 = 1000

Vậy không gian địa chỉ luận lý của P2 là: 0 – 1000.

(đối chiếu hình vẽ)



b. Xác định physical address của các lệnh A1, B1 của P1.

Áp dụng công thức 1, ta có:

+ Tính A1: 500 + 1000 = 1500

+ Tính B1: 700 + 1000 = 1700

c. Xác định physical address của các lệnh: A2, B2 của P2

Áp dụng công thức 1, ta có:

+ Tính A2: 250 + 3000 = 3250

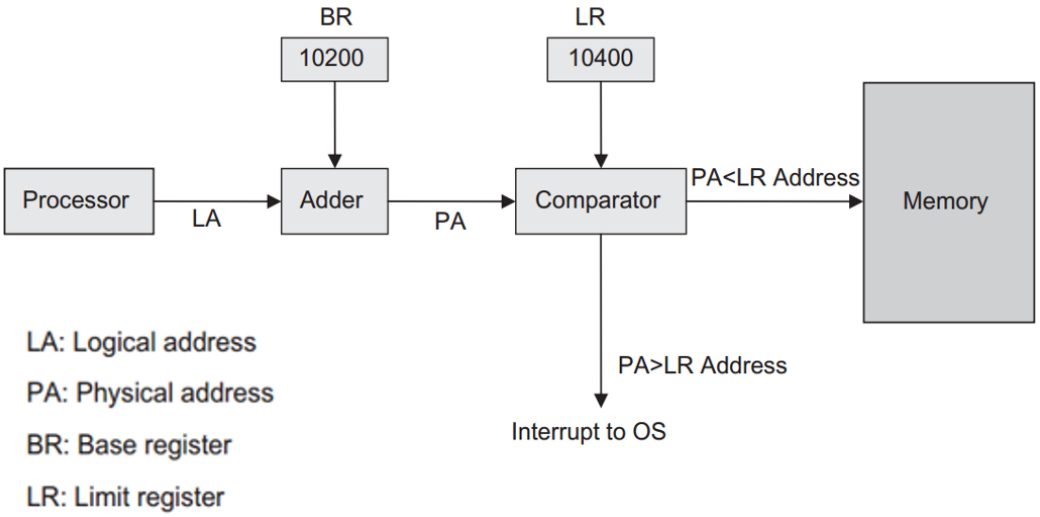
+ Tính B2: 600 + 3000 = 3600

*** Lưu ý**:

- Các địa chỉ luận lý và vật lý tính được đều nằm trong phạm vi hợp lệ.

- Sau khi chuyển đổi địa chỉ, nhớ phải kiểm tra để có kết luận đúng (hợp lệ hay không).

Sau đây là mô hình tổng kết việc chuyển đổi bộ nhớ: Giả thiết BR=10200; LR=10400.



Hình 5-4: Mô hình chuyển đổi địa chỉ ô nhớ.

Ví dụ 2:

Tính physical address của lệnh A (thuộc xử lý P1) biết rằng:

Lệnh A tại logical address: 800

Base register của P1 là: 2500

Limit register của P1 là 3200

Giải:

Physical address của lệnh A là: 2500 + 800 = 3300

Nhận thấy rằng: physical address của lệnh A > LR

Kết luận: Lệnh A có địa chỉ vật lý nằm ngoài không gian địa chỉ vật lý của P1

→ Physical address của lệnh A không hợp lệ.

Ví dụ 3:

Tính logical address của lệnh A (thuộc xử lý P1) biết rằng:

Lệnh A tại physical address: 3100

Limit register của P1 là: 3000

Không gian logical address của P1 là: 0 → 1000

Giải:

Base Register của P1: 3000 – 1000 = 2000

Logical address của lệnh A là: 3100 - 2000 = 1100

Địa chỉ logic của lệnh A không hợp lệ (vì 1100 > 1000)

3. Dynamic Loading

Chỉ khi nào được gọi đến, một thủ tục mới được load vào bộ nhớ nhằm tăng hiệu năng của bộ nhớ. Chức năng này rất hiệu quả trong trường hợp mã chương trình chiếm khối lượng lớn và không thường xuyên được sử dụng.

Cơ chế này không cần sự hỗ trợ đặc biệt của HĐH. HĐH chủ yếu cung cấp thư viện hỗ trợ, tạo điều kiện thuận lợi cho lập trình viên khai thác.

4. Overlay

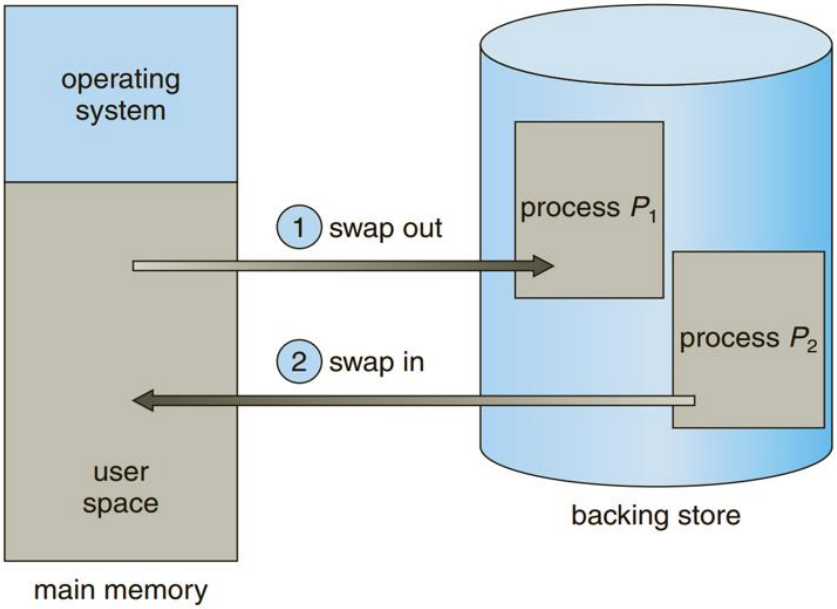
Tại mỗi thời điểm, hệ thống chỉ giữ lại trong bộ nhớ những lệnh hoặc dữ liệu cần thiết, giải phóng các lệnh hoặc dữ liệu chưa hoặc không cần đến. Chức năng này rất hữu dụng khi kích thước của tiến trình lớn hơn không gian bộ nhớ cấp cho tiến trình đó.

Chức năng này được điều khiển bởi người lập trình (thông qua sự hỗ trợ của các thư viện lập trình) chứ không cần sự hỗ trợ của HĐH.

5. Swapping

Một tiến trình có thể tạm thời bị swap ra khỏi bộ nhớ (RAM) và lưu trên một hệ thống lưu trữ phụ (HDD) nhằm tạo điều kiện cho các tiến trình khác thực thi. Sau đó, tiến trình (trên HDD) có thể được nạp lại vào bộ nhớ để tiếp tục quá trình thực thi.

Chức năng này có tính đến độ ưu tiên của tiến trình. Tiến trình có độ ưu tiên thấp hơn có thể bị swap-out để nhường chỗ cho tiến trình có độ ưu tiên cao hơn được swap-in vào bộ nhớ để thực thi.



Hình 5-5: Mô hình Swap-In và Swap-Out.

Ví dụ 4:

Process P1 có kích thước 200MB, có độ ưu tiên 1 cần được swap-in vào bộ nhớ để thực thi nhưng không đủ bộ nhớ. Process P2 đang đợi trong bộ nhớ có kích thước 250MB, có độ ưu tiên 10. P2 cần được swap-out.

Yêu cầu:

Tính thời gian cần để swap-out P2 và swap-in P1 là bao nhiêu nếu biết:

- Thời gian trễ (latency time) trung bình của HDD là 10 ms
- Tốc độ chuyển dời của HDD là 60 MB/s

Giải:

Thời gian chuyển dời của P2 là: 250/60 = 4.17 (s) = 4170 ms

Thời gian swap-out của P2 là: 4170 + 10 = 4180 ms

Thời gian chuyển dời của P1 là: 200/60 = 3.34 (s) = 3340 ms

Thời gian swap-in của P1 là: 3340 + 10 = 3350 ms

Tổng thời gian swap-out và swap-in của P2 và P1 là:

3350 + 4180 = 7530 ms (= 7.53 s)

6. Cấp phát bộ nhớ

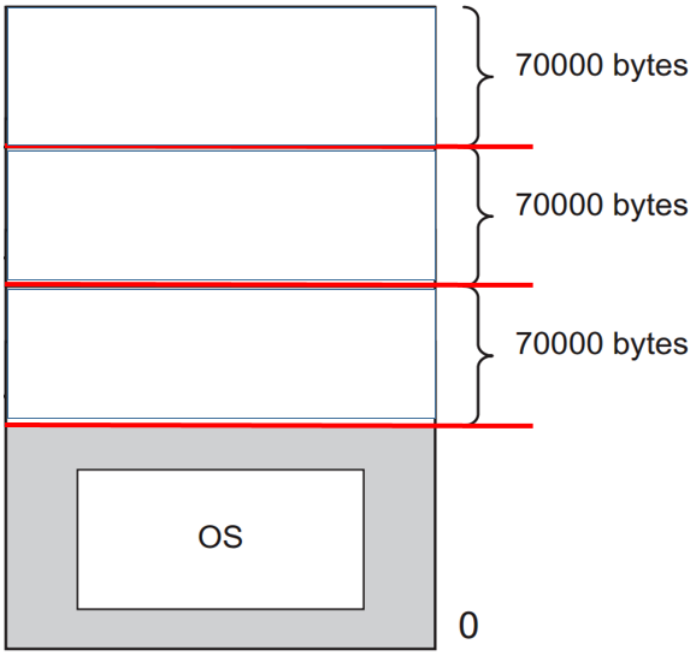
Trong phần này, chúng ta đề cập đến 2 cách cấp phát bộ nhớ là:

- **Static partition (Fixed-partition)**
- **Dynamic partition (Variable-partition)**

6.1. Static partition

Là cách chia bộ nhớ thành các phân đoạn (partitions) có ***kích thước cố định bằng nhau***. Mỗi phân đoạn chỉ chứa 1 process. Kích thước của mỗi phân đoạn có thể bằng nhau hoặc khác nhau. Cách này khá cũ, hiện nay rất ít được sử dụng.

Sau đây là hình ảnh bộ nhớ được chia trước thành các phân đoạn bằng nhau, mỗi đoạn đều có kích thước 70000 bytes.



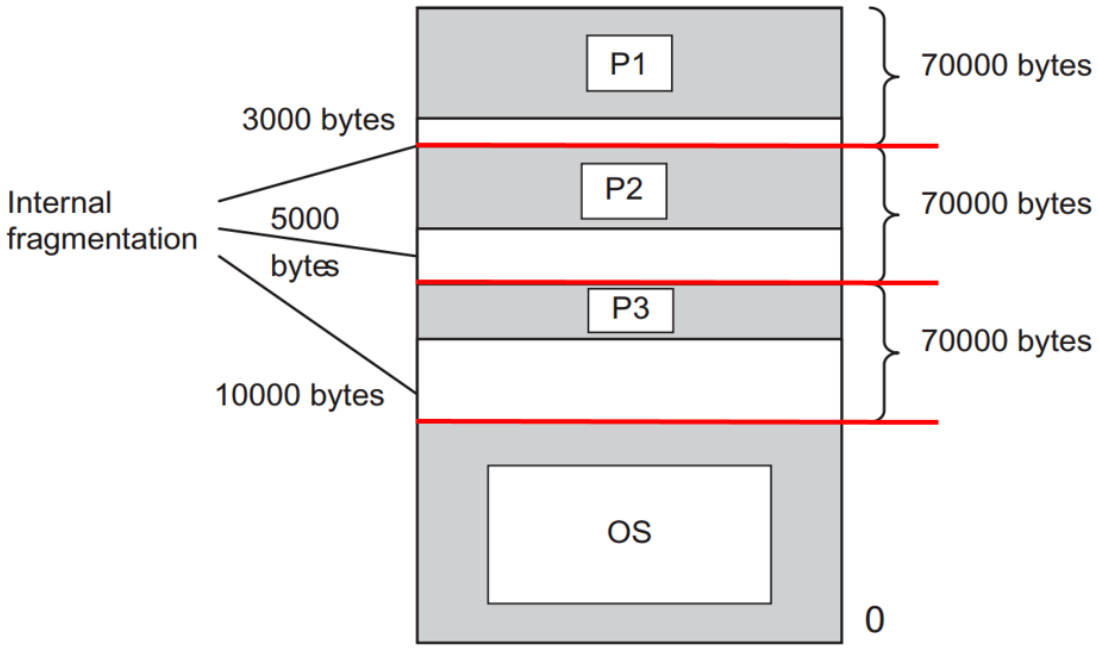
Hình 5-6: Bộ nhớ được chia thành các đoạn bằng nhau.

Giả sử có 3 xử lý được cấp phát vùng nhớ như sau: (hình 5-7)

P1: cần 67000 bytes

P2: cần 65000 bytes

P3: cần 60000 bytes



Hình 5-7: Bộ nhớ được cấp phát cho 3 xử lý.

Sau đó xử lý P4 đến, cần 7000 bytes bộ nhớ.

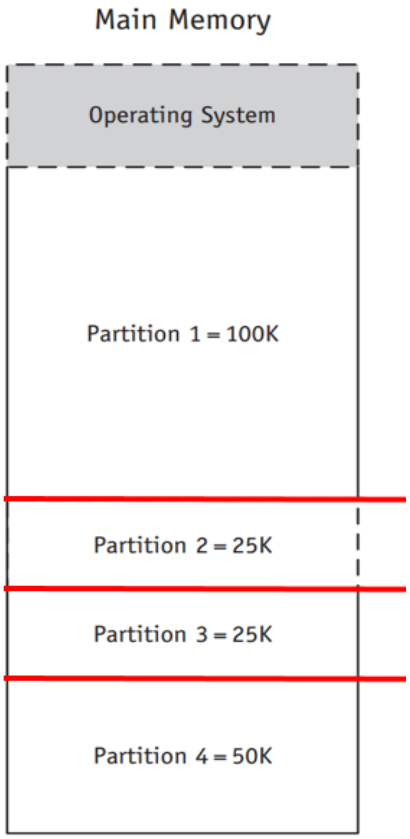
Hệ thống sẽ báo rằng đã hết bộ nhớ và P4 phải đợi. Tuy nhiên, từ hình 5-7, ta thấy rằng có 3 vùng nhớ còn trống trong mỗi phân đoạn nhưng hệ thống không thể sử dụng để cấp cho P4.

Các vùng trống này gọi là phân mảnh trong (Internal Fragmentation).

Việc cấp phát vùng nhớ tĩnh có thể dẫn đến phân mảnh trong.

Một cách cấp phát vùng nhớ tĩnh khác là chia bộ nhớ thành các phân đoạn (partitions) có ***kích thước cố định khác nhau***. Với cách chia này, các vùng nhớ có thể có kích thước khác nhau; Tuy nhiên, sau khi chia xong, chúng trở nên cố định.

Sau đây là hình ảnh cho thấy cách chia bộ nhớ thành các phân đoạn cố định có kích thước khác nhau:



Hình 5-8: Bộ nhớ được thành các phần cố định, kích thước khác nhau.

Giả thiết có 4 xử lý (Job) xuất hiện theo thứ tự và có nhu cầu bộ nhớ như sau:

Job 1: 30 KB

Job 2: 50 KB

Job 3: 30 KB

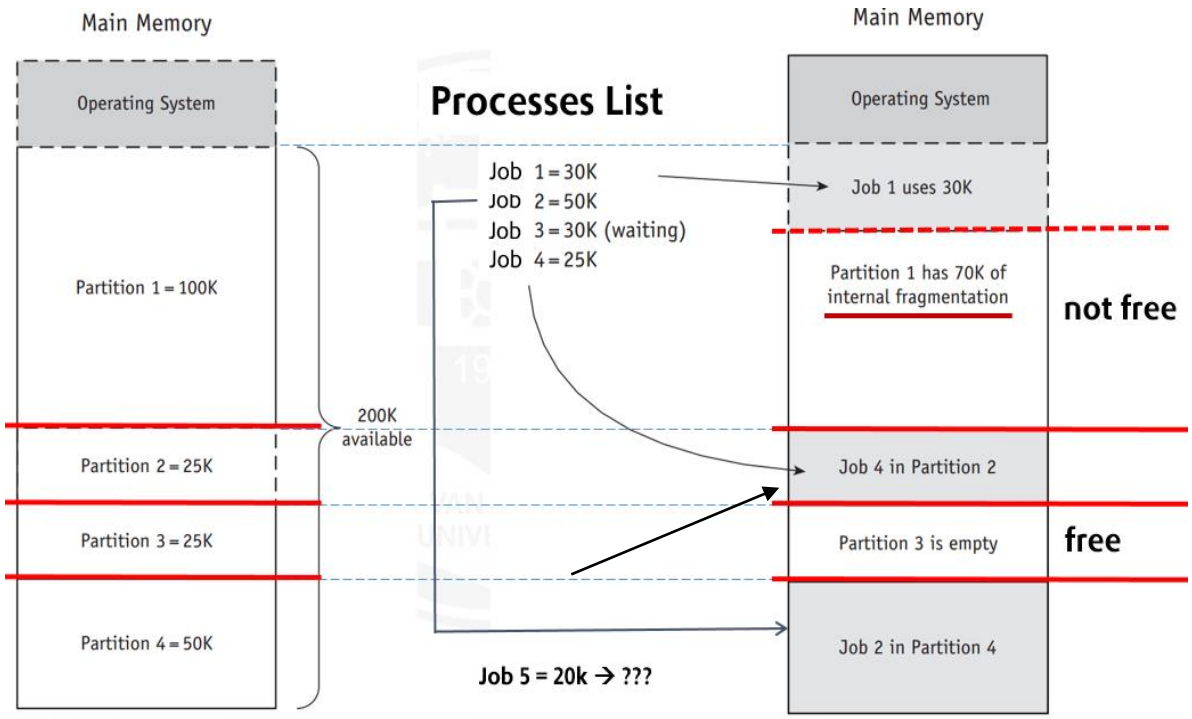
Job 4: 25 KB

Khi đó, Job 1 sẽ được cấp partition 1, Job 2 sẽ được cấp partition 4 (vì partition 2 và partition 3 không đủ), Job 3 không được cấp vùng nhớ vì không không còn vùng nhớ đủ để cấp, Job 4 được cấp partition 2.

Partition 3 vẫn còn trống (free) và Job 3 vẫn phải đợi.

Giả sử tiếp đó có Job 5 xuất hiện và có nhu cầu vùng nhớ là 20 KB thì partition 3 sẽ cấp cho Job 5.

Hình vẽ sau đây cho thấy các hoạt động vừa đề cập.



Hình 5-9: Hoạt động cấp phát vùng nhớ có kích thước cố định khác nhau.

*** Nhận xét**:

- Vẫn xuất hiện phân mảnh trong
- Không cải thiện được nhiều so với trường hợp kích thước bằng nhau

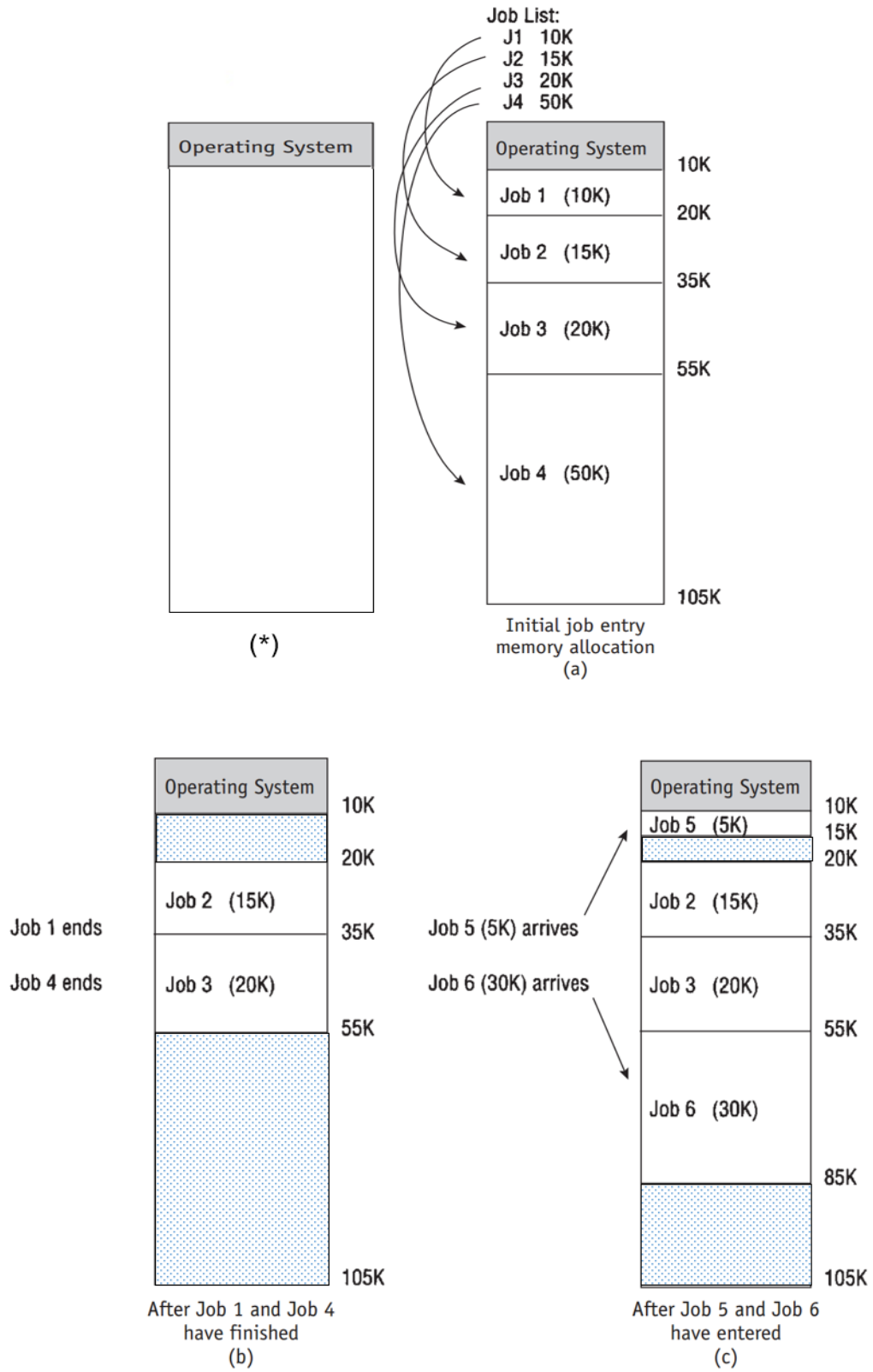
6.2. Dynamic partition

Là cơ chế cấp đủ lượng bộ nhớ theo nhu cầu của các tiến trình. Khi có nhiều tiến trình kết thúc và nhiều tiến trình được load vào bộ nhớ, sau một thời gian bộ nhớ cũng sẽ tạo ra sự phân mảnh; đó gọi là phân mảnh ngoài (external fragmentation).

Tuy nhiên, với cách cấp phát bộ nhớ động, bộ nhớ được sử dụng hiệu quả nhiều so với các cơ chế cấp phát tĩnh.

Hình ảnh sau đây cho thấy một hoạt động của cơ chế cấp phát bộ nhớ động; Trong hình này có một số ký hiệu:

- Các xử lý gọi là các Jobs
- Nền kẻ sọc là vùng nhớ còn trống
- Nền trắng là vùng nhớ đã được cấp



Hình 5-10: Hoạt động cấp phát vùng nhớ động.

*** Nhận xét:**

- Xuất hiện phân mảnh ngoài
- Các phân mảnh này vẫn có thể cấp cho tiến trình nếu có nhu cầu bộ nhớ phù hợp.

****

CÂU HỎI ÔN TẬP

Câu 1:

Công việc nào sau đây không phải thuộc trách nhiệm của quản lý bộ nhớ?

a. Cấp phát bộ nhớ cho các tiến trình
b. Ngăn chặn tiến trình có nhu cầu bộ nhớ quá nhiều
c. Kiểm tra việc truy xuất bộ nhớ hợp lệ
d. Kết gán địa chỉ bộ nhớ luận lý của user với địa chỉ bộ nhớ vật lý của hệ thống

Câu 2:

Mỗi ô nhớ trong bộ nhớ có thể có:

a. Một địa chỉ
b. Hai địa chỉ
c. Ba địa chỉ
d. Bốn địa chỉ

Câu 3:

Phát biểu nào sau đây là sai?

a. Không gian địa chỉ vật lý luôn lớn hơn không gian địa chỉ luận lý.
b. Không gian địa chỉ luận lý luôn nhỏ hơn không gian địa chỉ vật lý
c. Không gian địa chỉ luận lý có kích thước không cố định
d. Không gian địa chỉ luận lý có thể bằng không gian địa chỉ vật lý

Câu 4:

Phát biểu nào sau đây đúng với trường hợp dynamic loading?

a. Tại mỗi thời điểm chỉ giữ lại trong bộ nhớ những lệnh hoặc dữ liệu cần thiết, giải phóng các lệnh hoặc dữ liệu chưa hoặc không cần đến.
b. Một process có thể tạm thời bị swap ra khỏi bộ nhớ (RAM) và lưu trên một hệ thống lưu trữ phụ (HDD) nhằm tạo điều kiện cho process khác thực thi.
c. Chỉ khi nào được gọi đến thì một thủ tục mới được load vào bộ nhớ nhằm tăng hiệu năng của bộ nhớ.
d. Tất cả các câu trên đều sai.

Câu 5:

Việc chuyển đổi địa chỉ nhớ giữa địa chỉ vật lý và địa chỉ luận lý phụ thuộc vào:

a. Giá trị của 2 thanh ghi: BR và LR.
b. Kích thước tương ứng giữa 2 loại vùng nhớ.
c. Kích thước vùng nhớ đệm dùng để phục vụ trong quá trình chuyển đổi.
d. Số lượng tt đang hoạt động hiện tại.

*****

ĐÁP ÁN

Câu 1: b

Câu 2: b

Câu 3: d

Câu 4: c

Câu 5: a

*****

Chương 6: QUẢN LÝ LƯU TRỮ

1. Một số khái niệm

Files:

+ Là đơn vị lưu trữ thông tin, được lưu trên thiết bị lưu trữ như: HDD, USB, Memory Card, …
+ Có nhiều kiểu file: file văn bản, file hình ảnh, file video, …
+ Có thể là file có cấu trúc, có thể là file không cấu trúc, …

Thuộc tính của files: Mỗi file thường có nhiều thuộc tính:

- Archive: lưu trữ
- Hidden: ẩn
- Read-only: chỉ đọc
- System: thuộc về hệ thống
- …

Quyền trên files: Mỗi file thường được gán quyền để truy cập:

- Read
- Write
- Execute
- …

Các thao tác trên files: Sau đây liệt kê một số thao tác trên files:

- Create a file
- Write a file
- Saving a file
- Deleting a file
- Open a file
- List files

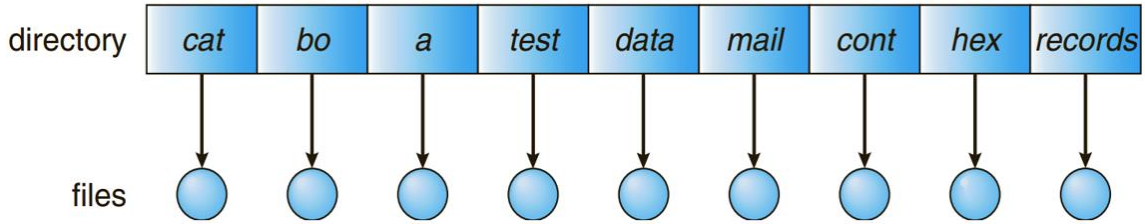
- Close a file
- Read a file
- Append a file
- Repositioning the current position pointer
- Get/Set attributes

Thư mục:

+ Là lập hợp gồm một hay nhiều files được nhóm lại với nhau để người dùng dể quản lý; Có thư mục không chứa file nào.
+ Thư mục có thể chứa thư mục con.
+ Bản chất là một vùng chứa (các files hay các thư mục con).

Kiến trúc thư mục:

+ Thư mục một cấp (Single Level Directory)



Hình 6-1: Kiến trúc thư mục 1 cấp.

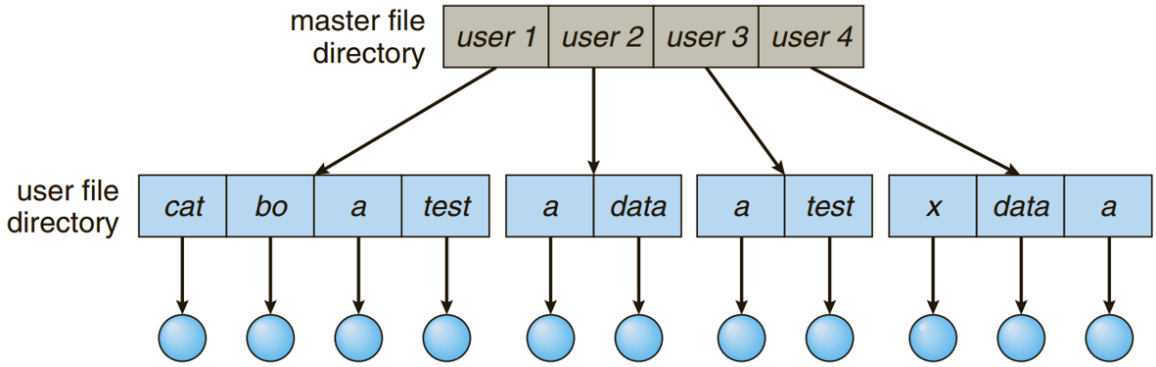
Ưu điểm:

- Đơn giản

Khuyết điểm:

- Dễ nhầm lẫn tên file (giữa các người sử dụng)
- Không thể phân nhóm file
- Khó khăn khi tìm kiếm file (nếu số lượng file nhiều)

+ Thư mục hai cấp (Two Level Directory)



Hình 6-2: Kiến trúc thư mục 2 cấp.

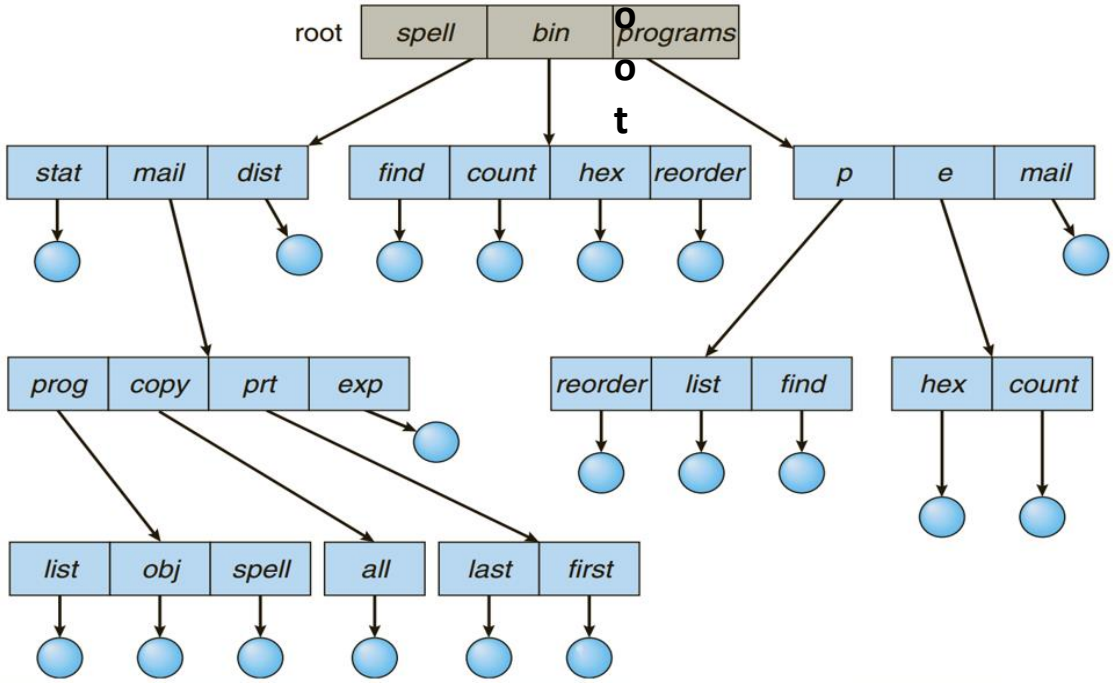
Ưu điểm:

- Mỗi user có thư mục riêng, tránh nhầm lẫn

Khuyết điểm:

- Không thể nhóm files
- Khó khăn khi tìm kiếm file (nếu số lượng file nhiều)

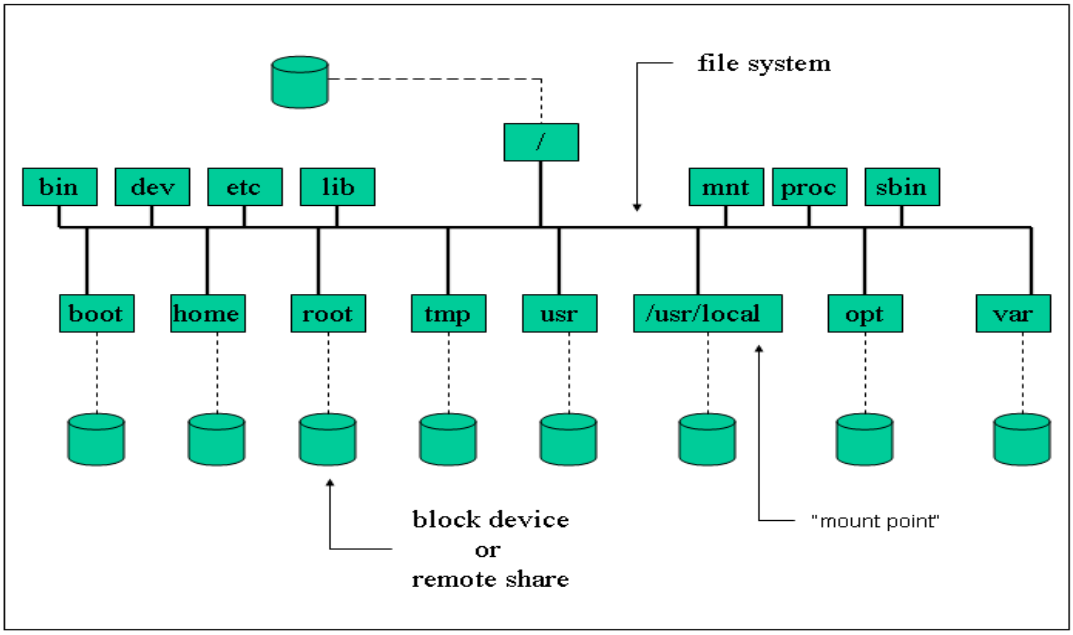
+ Thư mục đa cấp (Tree Structured Directory)



Hình 6-3: Kiến trúc thư mục đa cấp.

Kiến trúc này có một thư mục cấp cao nhất gọi là ***root*** chứa một số thư mục con. Mỗi user có thể tạo cho mình các thư mục con riêng; Trong mỗi thư mục con này có thể chứa files và các thư mục con khác.

Đây là cấu trúc thư mục hiện đại thông dụng nhất trong hệ thống máy tính. Một ví dụ của kiến trúc thư mục đa cấp là hệ thống tập tin và thư mục của Linux:



Hình 6-4: Kiến trúc thư mục trong Linux.

2. Kiến trúc đĩa cứng (HDD)

Trong phần này, chúng ta cùng làm việc với đĩa cứng (HDD).

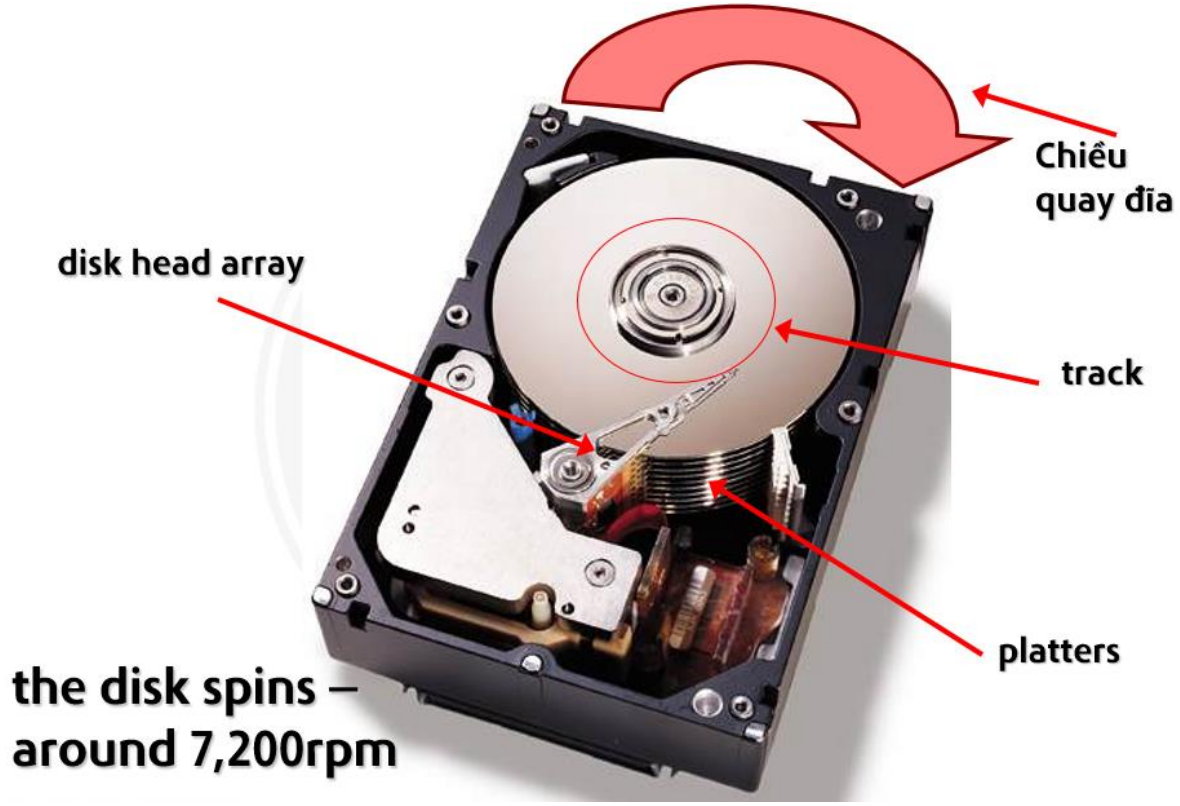
Trong môi trường đa nhiệm, nhiều xử lý đồng thời có yêu cầu I/O trên đĩa. CPU chỉ phục vụ 1 nhu cầu I/O tại 1 thời điểm; Do đó, các yêu cầu I/O đĩa khác phải đợi trong Disk Queue.

Vị trí cần truy cập trên đĩa có liên quan đến:

- Vị trí hiện tại của đầu đọc
- Hướng di chuyển của đầu đọc
- Tốc độ di chuyển của đầu đọc

Các yếu tố trên có ảnh hưởng đến độ ưu tiên trong quá trình lập lịch đĩa.

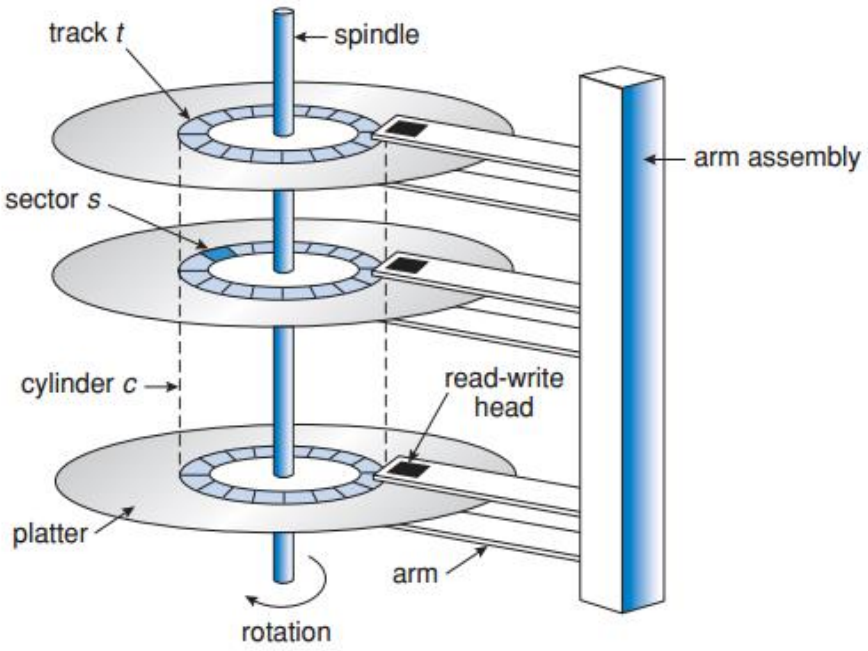
Xét hình ảnh cấu trúc của một HDD:



Hình 6-5: Kiến trúc HDD.

HDD bao gồm nhiều lá đĩa chồng lên nhau và liên tục quay tròn theo một chiều nhất định. Một đầu đọc/ghi có nhiệm vụ tiếp xúc và đọc ghi trên mỗi lá đĩa; Nó quay qua lại theo một cung ở một bên của lá đĩa.

Sau đây là hình ảnh phóng lớn của các lá đĩa và kiến trúc trên mỗi lá đĩa:



Hình 6-6: Kiến trúc lá đĩa.

Các thành phần trên các lá đĩa (HDD): Có 3 thành phần.

- Cylinders
- Tracks
- Sectors

3. Lập lịch truy xuất đĩa (HDD)

3.1. Điều kiện lập lịch đĩa: Dựa trên một số yếu tố sau.

+ **Seek Time**:

- Thời gian di chuyển đầu đọc để định vị đúng track/cylinder. Điều này phụ thuộc vào tốc độ hay cách di chuyển của đầu đọc.
- Ví dụ: Nếu disk head hiện đang ở vị trí cylinder 13, nó cần di chuyển đến cylinder 34 thì chiều dài di chuyển cùa nó là:

34 - 13 = 21 (lấy số lớn – số nhỏ)

- Cần xác định thời gian di chuyển trung bình (average seek time) của disk head.

+ **Rotational delay (latency: độ trễ)**: thời gian đầu đọc chờ đến đúng sector cần đọc. Điều này phụ thuộc tốc độ quay của đĩa.

+ **Transfer time**: thời gian chuyển dữ liệu từ đĩa vào bộ nhớ hoặc ngược lại. Điều này phụ thuộc băng thông kênh truyền giữa đĩa và bộ nhớ.

+ **Disk Bandwidth**: Bằng (Tổng số byte được chuyển dời) / (tổng thời gian từ lúc yêu cầu đến kết thúc công việc).

+ **Throughput**: Là tổng số yêu cầu được phục vụ trên một đơn vị thời gian.

+ **Response Time**: thời gian một yêu cầu cần phải đợi (trước khi được đáp ứng).

Ta có công thức:

Disk I/O time = Seek time + Rotational time + Transfer time

3.2. Các thuật toán lập lịch đĩa:

Có nhiều thuật toán lập lịch truy xuất đĩa. Trong chương này chúng ta cùng khảo sát 4 thuật toán:

- FCFS (First Come First Served)
- SSTF (Shortest-Seek-Time First)
- SCAN
- C-SCAN (Circular SCAN)

Các thuật toán này quan tâm đến quãng đường di chuyển trung bình của disk head. Giá trị này càng nhỏ càng tốt.

Trong phần này, chúng ta sử dụng cùng một ví dụ (giả thiết và yêu cầu) và lần lượt áp dụng các thuật toán khác nhau để qua đó chúng ta có thể so sánh giữa chúng.

3.2.1. Thuật toán FCFS

Đặc điểm:

- Các yêu cầu được lập lịch giải quyết theo thứ tự đứng trong disk queue.
- Có thể không tối ưu.

Ví dụ 1: Cho disk queue có I/O requests đến tại các cylinders: 54, 97, 73, 128, 15, 44, 110, 34, 45. Giả sử ban đầu disk head đang đứng tại cylinder 23.

Yêu cầu:

Áp dụng thuật toán FCFS:

a. Vẽ sơ đồ di chuyển của disk head.

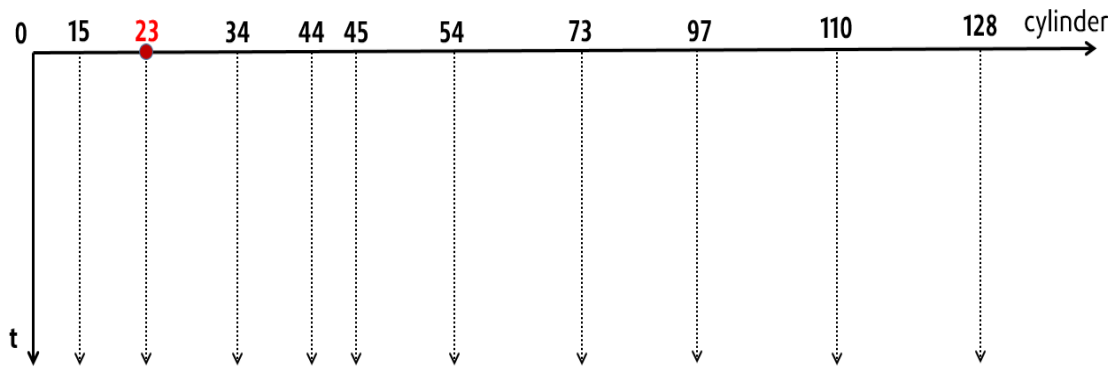
b. Tính tổng quảng đường di chuyển của disk head.

c. Tính quảng đường di chuyển trung bình của disk head.

Giải:

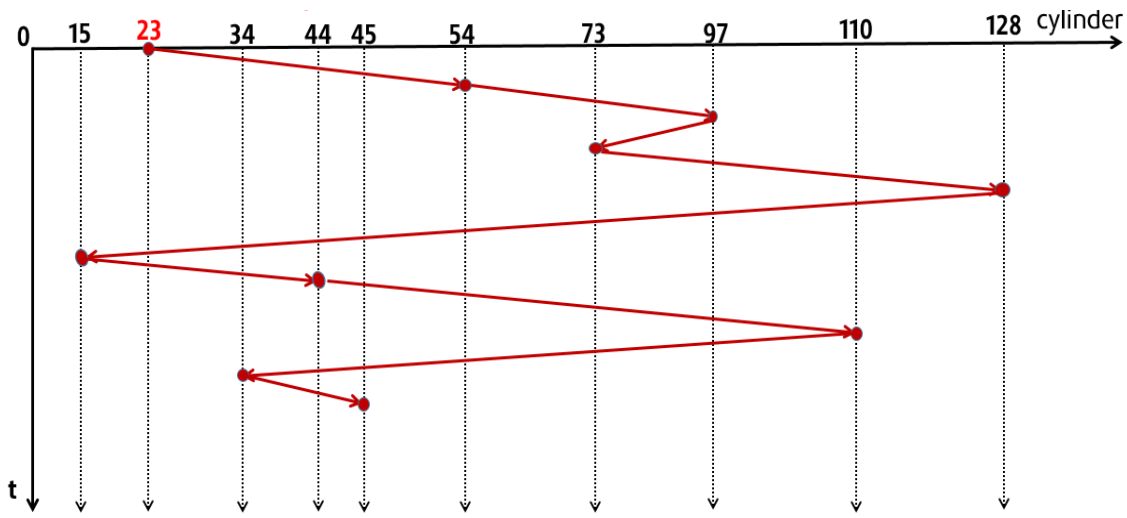
a. Vẽ sơ đồ di chuyển của disk head.

Đầu tiên ta vẽ hệ trục tọa độ như sau:



Hình 6-7: Tọa độ chuẩn bị cho sơ đồ di chuyển của đầu đọc.

Kế tiếp, vẽ vị trí truy xuất rồi đến đường di chuyển của đầu đọc.



Hình 6-8: Sơ đồ di chuyển của đầu đọc ghi.

Lưu ý:

- Dấu chấm trên cùng tại vị trí 23 thể hiện đầu đọc ghi đang đứng trước khi bắt đầu truy xuất đĩa.
- Các đầu chấm còn lại là các vị trí truy xuất đĩa.
- Mũi tên 1 chiều cho biết hướng di chuyển của đầu đọc và có xu hướng đi xuống theo chiều dương của trục thời gian (t).

b. Tính tổng quảng đường di chuyển của disk head.

Ta lập bảng như sau:

Vị trí truy xuất	Quãng đường	Số cylinders
54	23 → 54	31
97	54 – 97	43
73	97 – 73	24
128	73 – 128	55
15	128 – 15	113
44	15 – 44	29
110	44 – 110	66
34	110 – 34	76
45	34 – 45	11
	TỔNG	**448** (cylinders)

Như vậy, tổng quãng đường di chuyển của đầu đọc là: 448 cylinders.

Lưu ý cách tính: Trong bảng có 3 cột:

- **Cột vị trí truy xuất**: chứa các vị trí cần truy xuất (cylinders) theo đúng trình tự xuất hiện trong hàng đợi.
- **Cột quãng đường**: chứa số cylinders mà đầu đọc ghi phải đi qua để đến được vị trí truy xuất kế tiếp; Thường lấy số lớn trừ số nhỏ.
- **Cột số cylinders**: chứa kết quả phép toán tương ứng từ cột quãng đường.

c. Tính quảng đường di chuyển trung bình của disk head.

448 / 9 ≈ 49.8 (cylinders)

3.2.2. Thuật toán SSTF

Đặc điểm:

- **Seek Time**: Thời gian đầu đọc di chuyển đến đúng vị trí.
- Yêu cầu nào có seek time ngắn nhất sẽ được thực thi trước.
- HĐH tính toán **seek time** của tất cả yêu cầu trong hàng đợi để chọn ra yêu cầu thích hợp.
- Các yêu cầu gần với vị trí đầu đọc hiện tại sẽ được ưu tiên phục vụ trước.
- Các yêu cầu xa so với vị trí đầu đọc hiện tại sẽ chờ.

Ví dụ 2: Cho disk queue có I/O requests đến tại các cylinders: 54, 97, 73, 128, 15, 44, 110, 34, 45. Giả sử ban đầu disk head đang đứng tại cylinder 23.

(như ví dụ 1)

Yêu cầu:

Áp dụng thuật toán SSTF:

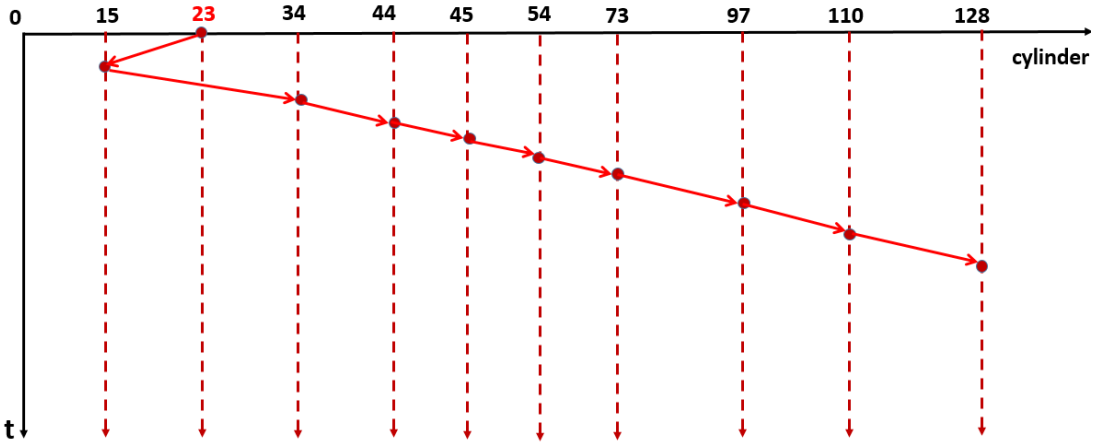
a. Vẽ sơ đồ di chuyển của disk head.

b. Tính tổng quảng đường di chuyển của disk head.

c. Tính quảng đường di chuyển trung bình của disk head.

Giải:

a. Vẽ sơ đồ di chuyển của disk head.



b. Tính tổng quảng đường di chuyển của disk head.

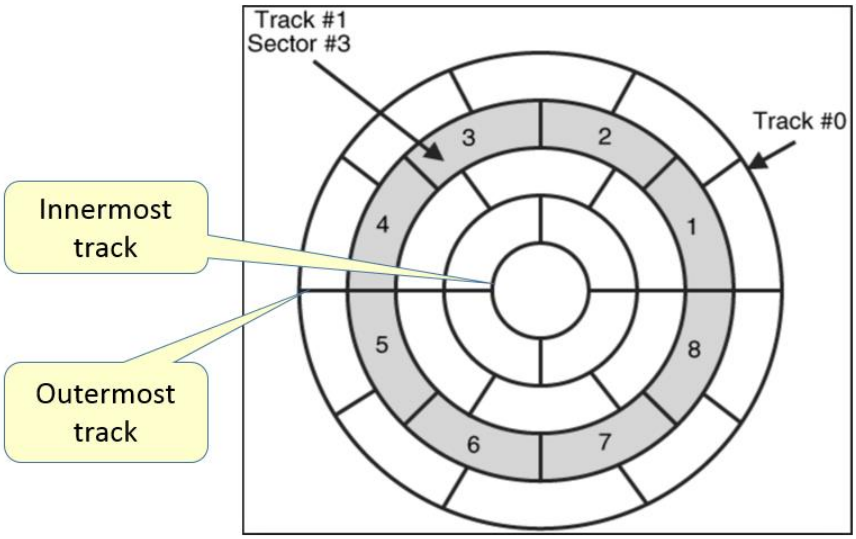
Vi trí truy xuất (cylinder)	Quãng đường	Chiều dài
15	23 – 15	8
34	15 – 34	19
44	34 – 44	10
45	44 – 45	1
54	45 – 54	9
73	54 – 73	19
97	73 – 97	24
110	97 – 110	13
128	110 – 128	18
	TỔNG	**121 (cylinders)**

c. Tính quảng đường di chuyển trung bình của disk head.

$$S_{tb} = 121/9 \approx 13.44 \text{ (cylinders)}$$

3.2.3. Thuật toán SCAN

Chúng ta xem lại hình ảnh của lá đĩa và hướng di chuyển của đầu đọc dựa trên hình sau:



Hình 6-9: Các quy ước trên lá đĩa.

Đặc điểm:

- Đầu đọc di chuyển theo 1 hướng, ví dụ từ ***outermost*** đến ***innermost*** rồi sau đó di chuyển theo hướng ngược lại.
- Hướng giảm của số cylinders là hướng từ ngoài vào trong.
- Các yêu cầu được thực thi theo hướng di chuyển của đầu đọc.
- Sau đó các yêu cầu theo hướng di chuyển ngược lại sẽ được thực hiện sau.

Ví dụ 3:

Disk queue với I/O requests đến theo thứ tự tại các cylinder sau: 6, 10, 12, 54, 97, 73, 128, 15, 44, 110, 34, 45.

Giả sử đầu đọc hiện ở tại cylinder 23 và **di chuyển theo hướng giảm của số cylinder**.

HDD có **tổng cộng 150 cylinders**.

Yêu cầu: Áp dụng thuật toán SCAN:

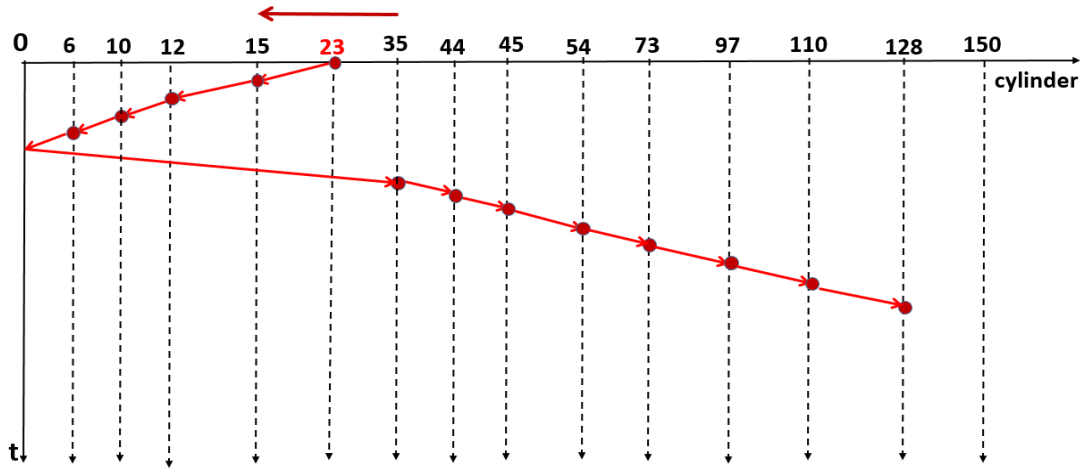
a. Vẽ sơ đồ di chuyển của disk head.

b. Tính tổng quảng đường di chuyển của disk head.

c. Tính quảng đường di chuyển trung bình của disk head.

Giải:

a. Vẽ sơ đồ di chuyển của disk head.



b. Tính tổng quảng đường di chuyển của disk head.

Vị trí truy xuất	Quảng đường	Khoảng cách
15	23 – 15	8
12	15 – 12	3
10	12 – 10	2
6	10 – 6	4
0	**6 – 0**	6
35	**0 – 35**	35
44	35 – 44	9
45	44 – 45	1
54	45 – 54	9
73	54 – 73	19
97	73 – 97	24
110	97 – 110	13
128	**110 - 128**	18
	TỔNG	**151 (cylinders)**

c. Tính quảng đường di chuyển trung bình của disk head.

$$S_{tb} = 151/13 = 11.61 \text{ (cylinders)}$$

3.2.4. Thuật toán C-SCAN

Đặc điểm:

- Khắc phục một số nhược điểm của giải thuật SCAN.
- Đầu đọc thực hiện các yêu cầu truy xuất địa theo 1 chiều (vòng tròn).
- Sau khi đầu đọc di chuyển đến cylinder 0 (từ inner đến outter), nó quay lại cylinder 150 (không thực hiện bất kỳ yêu cầu nào trên đường quay lại) và thực hiện các yêu cầu khác theo hướng cùng hướng với lần di chuyển trước.

Ví dụ 4:

Disk queue với I/O requests đến theo thứ tự tại các cylinder sau: 6, 10, 12, 54, 97, 73, 128, 15, 44, 110, 34, 45.

Giả sử đầu đọc hiện ở tại cylinder 23 và **di chuyển theo hướng giảm của số cylinder**. HDD có **tổng cộng 150 cylinders**.

Yêu cầu:

Áp dụng thuật toán C-SCAN:

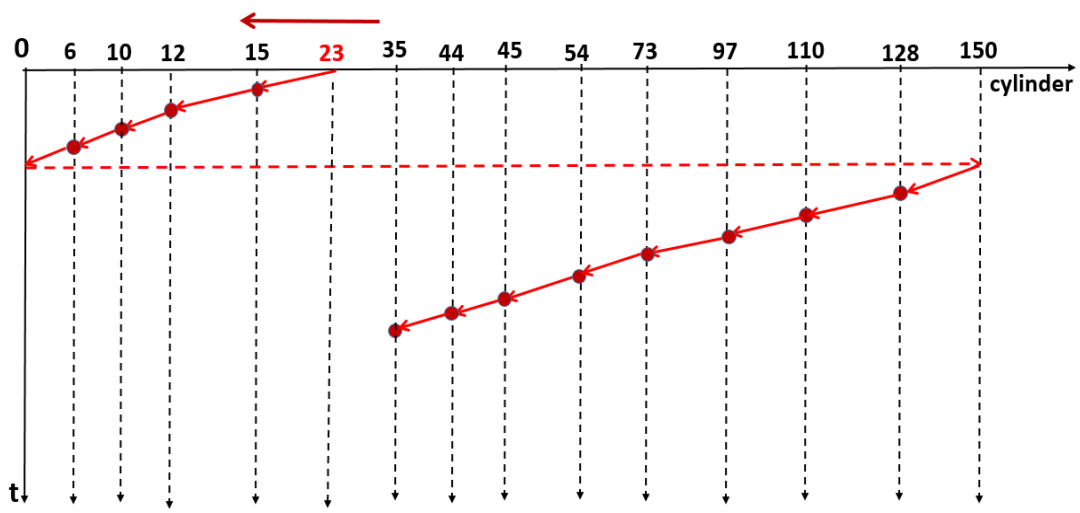
a. Vẽ sơ đồ di chuyển của disk head.

b. Tính tổng quảng đường di chuyển của disk head.

c. Tính quảng đường di chuyển trung bình của disk head.

Giải:

a. Vẽ sơ đồ di chuyển của disk head.



b. Tính tổng quảng đường di chuyển của disk head.

Vị trí truy xuất	Quảng đường	Khoảng cách
15	23 – 15	8
12	15 – 12	3
10	12 – 10	2
6	10 – 6	4
0	**6 – 0**	6
150	**0 → 150**	**0**
128	150 – 128	22
110	128 – 110	18
97	110 – 97	13
73	97 – 73	24
54	73 – 54	19
45	54 – 45	9
44	**45 – 44**	1
35	44 – 35	9
	TỔNG	**138 (cylinders)**

c. Tính quảng đường di chuyển trung bình của disk head.

$$S_{tb} = 138/13 \approx 10.61 \text{ (cylinders)}$$

*****

CÂU HỎI ÔN TẬP

Câu 1:

Việc đánh số cylinders trên lá đĩa của HDD theo qui ước:

a. Ngẫu nhiên, tùy vào hệ thống
b. Từ trong ra ngoài
c. Từ ngoài vào trong
d. Theo hướng di chuyển của đầu đọc

Câu 2:

Thành phần nào sau đây không phải là đơn vị phân chia trên HDD?

a. Sector
b. Cylinder
c. Track
d. Volumn

Câu 3:

Seek Time là thời gian di chuyển của đầu đọc:

a. Đến vị trí truy xuất
b. Đến vị trí truy xuất từ vị trí kế trước đó
c. Từ lúc bắt đầu truy xuất đến vị trí truy xuất
d. Từ vị trí truy xuất đến vị trí kế tiếp

Câu 4:

Phát biểu nào sau đây đúng về CPU phục vụ truy xuất đĩa trong môi trường đa nhiệm?

a. CPU chỉ phục vụ 1 nhu cầu I/O tại 1 thời điểm.
b. CPU phục vụ yêu cầu nào mà nó thấy thuận tiện nhất
c. Có thể buộc 1 CPU phục vụ cùng lúc nhiều yêu cầu bằng giải pháp phần mềm
d. Vì là hệ thống đa nhiệm nên CPU phục vụ cùng lúc nhiều nhu cầu I/O đĩa

Câu 5:

Cấu trúc tập tin và thư mục trong các hệ thống hiện đại hiện nay là?

a. Single-Level Directory.
b. Two-Level Directory
c. Tree-Level Directory
d. Multi-Tree-Level Directory

*****

ĐÁP ÁN

Câu 1: c

Câu 2: d

Câu 3: b

Câu 4: a

Câu 5: c

*****

TÀI LIỆU THAM KHẢO

[1]. Hồ Đắc Phương, *Giáo Trình Nguyên Lý Hệ Điều Hành*, NXB Giáo dục Việt Nam.

[2]. Andrew S. Tanenbaum, Herbert Bos, *Modern Operating Systems*, Pearson, 2014.

[3]. Abraham Silberschatz, *Operating System Concepts*, 10th edition, Wiley 2018.

*****